A project report on

MACROGRAPH AI

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

HARIPRASAADH K (23BAI1061)
SJ ARULPRASAAD (23BAI1089)
NITHISH KUMAR CS (23BAI1035)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2026

MACROGRAPH AI

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

HARIPRASAADH K (23BAI1061)

SJ ARULPRASAAD (23BAI1089)

NITHISH KUMAR CS (23BAI1035)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2026

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled "MACROGRAPH AI" submitted by NITHISH KUMAR CS (23BAI1035), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Dr. Tulasi Prasad Sariki.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: **Signature of the Candidate**



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "MACROGRAPH AI" is prepared and submitted by NITHISH KUMAR CS (23BAI1035) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning) is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr. Tulasi Prasad Sariki

Date:

Signature of the Examiner	Signature of the Examiner
Name:	Name:
Date:	Date:

Approved by the Head of Department,
Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

Name: Dr. Tulasi Prasad Sariki

Date:

(Seal of School of Computer Science and Engineering)

ABSTRACT

Analysing the Indian economy requires data from many agencies that publish in different formats, at different frequencies and with different definitions. Language models can write fluent economic narratives, but they can also state figures that appear in no official release, which makes unsupported output unacceptable for economic analysis. This project presents Macrograph AI, a multi-agent platform for Indian macroeconomic intelligence in which every number must be traceable to an official source.

The economy is decomposed into ten sector agents: real sector, agriculture, prices, monetary conditions, finance, capital markets, fiscal affairs, external sector, labour and services. Each agent owns its indicators and obtains them only through its own FastMCP data server, with a fallback from live data to a cached snapshot to an explicit unavailable status. The agents cooperate through a native typed agent-to-agent protocol that enforces seven declared dependencies, rejects loops and refuses successful responses without source provenance. A four-node LangGraph orchestrator routes questions over agent cards, collects observations, traverses a causal knowledge graph of twenty indicators and twelve typed edges, runs an optional scenario simulation and produces a report in which each observation carries its period, status and a provenance hash. Research instruments include an edge evidence audit, an anomaly radar, trajectory and twin views, a simulated monetary policy committee, a daily brief and a claim verification library that recomputes numerical claims against content-addressed evidence. A web application provides a cited dashboard, streaming chat, agent registry and causal laboratory.

The implementation of about 37,000 lines of Python (excluding tests) and 11,300 lines of TypeScript is covered by 455 test functions; the complete run passed 550 of 551 cases. An end-to-end run answered a cross-sector crude oil question with live official data in about eleven seconds. The platform's own audit found that none of the twelve causal edges can yet be confirmed from the available series, and that the scenario engine is a transparent heuristic, so these elements are reported as assumptions and simulations. Connecting claim verification to the answers, estimating the causal edges from long series and evaluating the system against baselines are identified as the next steps.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Dr. Tulasi Prasad Sariki, HoD B.Tech - Artificial Intelligence and Machine Learning, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for their constant guidance, continual encouragement, understanding; more than all, they taught me patience in my endeavor. My association with them is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of Artificial Intelligence and Machine Learning.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to Dr. Tulasi Prasad Sariki, Head of the Department, B.Tech. Computer Science and Engineering (Artificial Intelligence and Machine Learning) and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: **Nithish Kumar CS**

CONTENTS

CONTENTS v

LIST OF FIGURES viii

LIST OF TABLES x

LIST OF ACRONYMS xi

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION 1

1.2 OVERVIEW OF MACROECONOMIC INTELLIGENCE SYSTEMS 2

1.3 CHALLENGES PRESENT IN AUTOMATED MACROECONOMIC ANALYSIS 2

1.4 PROJECT STATEMENT 3

1.5 OBJECTIVES 4

1.6 SCOPE OF THE PROJECT 5

1.7 CONTRIBUTIONS OF THE WORK 6

1.8 ORGANISATION OF THE REPORT 6

CHAPTER 2
BACKGROUND

2.1 INTRODUCTION 8

2.2 SURVEY ON LLM-BASED AGENTS FOR ECONOMICS AND FINANCE 8

2.3 RETRIEVAL-AUGMENTED GENERATION IN FINANCE 9

2.4 KNOWLEDGE GRAPHS FOR MACROECONOMICS 10

2.5 AGENT INTEROPERABILITY PROTOCOLS 11

2.6 CAUSALITY AND SCENARIO ANALYSIS IN MACROECONOMICS 12

2.7 TRUSTWORTHY GENERATION AND HALLUCINATION 13

2.8 COMPARATIVE SUMMARY OF RELATED WORK 13

2.9 RESEARCH GAPS AND MOTIVATION 14

2.10 TECHNOLOGIES USED 14

CHAPTER 3
SYSTEM REQUIREMENTS AND DESIGN

3.1 INTRODUCTION 16

3.2 REQUIREMENTS ANALYSIS 16

3.3 SYSTEM CONTEXT 17

3.4 LAYERED ARCHITECTURE 18

3.5 DESIGN OF A SECTOR AGENT 19

3.6 INDICATOR OWNERSHIP AND DEPENDENCIES 20

3.7 DATA MODEL 21
3.8 DESIGN DECISIONS AND RATIONALE 22

CHAPTER 4
PROTOCOL DESIGN

4.1 INTRODUCTION 24
4.2 THE AGENT-TO-AGENT LAYER 24
4.3 AGENT CARDS, REGISTRY AND ROUTING 28
4.4 CROSS-SECTOR EXCHANGES 28
4.5 THE MCP DATA LAYER 29
4.6 PROVENANCE ACROSS THE PROTOCOLS 30
4.7 TRACEABILITY 31

CHAPTER 5
IMPLEMENTATION

5.1 INTRODUCTION 32
5.2 DEVELOPMENT ENVIRONMENT 32
5.3 THE ORCHESTRATOR 32
5.4 CHAT INTERFACE AND ROUTING 35
5.5 SECTOR AGENTS AND DATA ACQUISITION 35
5.6 STORAGE 37
5.7 ROBUSTNESS AND OFFLINE OPERATION 38
5.8 CODE ORGANISATION AND STATISTICS 38

CHAPTER 6
KNOWLEDGE GRAPH AND SCENARIO ENGINE

6.1 INTRODUCTION 39
6.2 ONTOLOGY OF INDICATORS AND TRANSMISSION EDGES 39
6.3 GRAPH ENGINE AND QUERIES 41
6.4 SCENARIO ENGINE 42
6.5 WORKED EXAMPLE: CRUDE OIL SHOCK 43

CHAPTER 7
RESEARCH LAB, DAILY BRIEF AND CLAIM VERIFICATION

7.1 INTRODUCTION 45
7.2 SHOCKWAVE: MONTHLY TRAJECTORIES 45
7.3 TWIN: BASE WORLD VERSUS SHOCKED WORLD 45
7.4 ANOMALY RADAR 46
7.5 EDGE EVIDENCE AUDIT 46
7.6 SIMULATED MONETARY POLICY COMMITTEE 48
7.7 CONSENSUS AND MODEL CARD 50
7.8 DAILY EARLY-WARNING BRIEF 51
7.9 CLAIM VERIFICATION 51

CHAPTER 8
USER INTERFACE AND API

8.1 INTRODUCTION 54
8.2 FRONTEND ARCHITECTURE 54
8.3 LANDING PAGE AND AUTHENTICATION 55
8.4 DASHBOARD 56
8.5 AGENT RESEARCH WORKSPACE 57
8.6 A2A AGENT REGISTRY 58
8.7 CAUSAL LAB 59
8.8 HTTP INTERFACE 61

CHAPTER 9
TESTING, RESULTS AND DISCUSSION

9.1 INTRODUCTION 62
9.2 TESTING STRATEGY 62
9.3 TEST RESULTS 62
9.4 END-TO-END ORCHESTRATOR RUN 64
9.5 EDGE EVIDENCE AUDIT RESULT 64
9.6 ANOMALY RADAR AND SIMULATED COMMITTEE 66
9.7 PLANNED EVALUATION 66
9.8 LIMITATIONS 67
9.9 DISCUSSION 67

CHAPTER 10
CONCLUSION AND FUTURE WORK

10.1 SUMMARY OF THE WORK 69
10.2 ACHIEVEMENT OF THE OBJECTIVES 69
10.3 CONTRIBUTIONS AND INSIGHTS 70
10.4 FUTURE WORK 70
10.5 CONCLUDING REMARKS 72

REFERENCES 73
APPENDICES 76
APPENDIX 1 REPOSITORY STRUCTURE AND CONFIGURATION 77
APPENDIX 2 CANONICAL INDICATORS OF THE KNOWLEDGE GRAPH 78
APPENDIX 3 SAMPLE REPORT GENERATED BY THE ORCHESTRATOR 79
APPENDIX 4 SELECTED SOURCE CODE LISTINGS 81
APPENDIX 5 ERROR TAXONOMY AND OPEN ISSUES 84

LIST OF FIGURES

1.1 Mapping of identified problems to project objectives and implemented components. 5
2.1 Separation of the agent-to-agent reasoning channel (A2A) and the agent-to-data channel (MCP). 12
3.1 System context of Macrograph AI: users, external data sources and external services. 18
3.2 Layered architecture of Macrograph AI. 19
3.3 Standard internal structure of a sector agent package. 20
3.4 Indicator ownership and the seven declared sector-to-sector A2A dependencies. 21
3.5 Entity-relationship diagram of the canonical economic data layer. 22
4.1 Validation pipeline applied by the A2A server to every request. 26
4.2 Sequence of an A2A delegation in which a sector agent requests a peer indicator. 28
4.3 An agent retrieving data through its own FastMCP server, with the fallback order. 30
4.4 Propagation of provenance from the official source to the final report. 31
5.1 The four-node LangGraph orchestrator and the state keys written by each node. 33
5.2 Sections of the cited intelligence report produced by the synthesis node. 35
5.3 Data acquisition pipeline shared by the sector agents. 36
5.4 Primary data sources feeding each sector agent. 37
6.1 Causal transmission graph of 20 indicators connected by 12 typed edges (lag in months and sign). 40
6.2 Structure, storage and query services of the knowledge graph. 42
6.3 Causal transmission graph and downstream uses of shock propagation. 43
6.4 Simulated monthly change from baseline after the shock (step response, produced by the Shockwave module). 44
7.1 Research lab modules, API endpoints and user interface tabs. 45
7.2 Per-hop share of attenuation along the path from Brent crude to GDP growth (Twin module, real output). 46
7.3 Edge evidence audit: seeded priors tested against live series. 48
7.4 Simulation of a six-seat Monetary Policy Committee with a deterministic rule baseline. 50
7.5 Claim verification pipeline from retrieved evidence to rendered text and audit record. 52
7.6 Proposed integration of the claim verifier into orchestrator synthesis (not yet implemented). 53
8.1 React component hierarchy of the application. 55
8.2 Landing page of Macrograph AI. 56

8.3 Dashboard, all-sectors tab with ticker bar and early-warning brief panel. 56
8.4 Dashboard, finance and banking tab: credit expansion, asset quality gauges and lending rates. 57
8.5 Server-Sent Events protocol between the chat workspace and the gateway. 58
8.6 A2A agent registry view listing the Agent Cards of the sector agents. 59
8.7 Causal lab, path tab: transmission path from the repo rate to GDP growth..... 60
8.8 Causal lab, Shockwave tab: animated propagation of a Brent crude shock. 60
8.9 Causal lab, MPC tab: simulated six-seat committee labelled as simulated. 61
9.1 Number of test functions per test area (455 functions). 63
9.2 Overlapping monthly observations available for each causal edge (minimum required: 10). 65
10.1 Target architecture after planned improvements. 72
A1.1 Repository layout of Macrograph AI. 77
A5.1 Taxonomy of A2A error codes. 84

LIST OF TABLES

1.1 Organisation of the report 7
2.1 Comparison of related work and Macrograph AI 13
2.2 Technologies used in Macrograph AI 15
3.1 Functional requirements 16
3.2 Non-functional requirements 17
3.3 Indicator ownership by sector agent 21
3.4 Principal design decisions and their rationale 23
4.1 Fields of the principal A2A messages 24
4.2 A2A error codes 27
4.3 Declared sector-to-sector A2A dependencies 29
5.1 Development and runtime environment 32
5.2 Source lines by component (tests excluded, 12 October 2026) 38
6.1 The twelve causal edges of the knowledge graph 41
6.2 Simulated transmission of a USD 20 per barrel Brent increase (default baselines) 43
8.1 Principal HTTP endpoints 61
9.1 Test functions by area (12 October 2026) 63
9.2 Observations collected in the end-to-end run 64
9.3 Result of the edge evidence audit 65
9.4 Votes of the simulated committee (rule-based mode) 66
10.1 Status of the project objectives 70
A1.1 Principal configuration variables 77
A2.1 Canonical indicators of the ontology 78
A3.1 Indicator matrix of the generated report 79
A3.2 Scenario table of the generated report (live baselines for CPI and repo rate) 79
A3.3 Source provenance table of the generated report 80
A4.1 Listing: A2A request and response models (core/protocols/a2a/messages.py) 81
A4.2 Listing: Shortest transmission path query (core/knowledge_graph/networkx_engine.py) 81
A4.3 Listing: Scenario propagation loop (core/econometrics/causal_engine.py) 82
A4.4 Listing: Rolling z-score of the anomaly radar (core/research/anomaly.py) 82
A5.1 Principal open issues from the code review 84

LIST OF ACRONYMS

A2A	Agent-to-Agent protocol
ADF	Augmented Dickey-Fuller test
API	Application Programming Interface
BH	Benjamini-Hochberg false discovery rate procedure
BoP	Balance of Payments
BVAR	Bayesian Vector Autoregression
CGA	Controller General of Accounts
CPI	Consumer Price Index
CSV	Comma-Separated Values
DBIE	Database on Indian Economy (Reserve Bank of India)
DFD	Data Flow Diagram
EPFO	Employees' Provident Fund Organisation
GDP	Gross Domestic Product
GFCF	Gross Fixed Capital Formation
GST	Goods and Services Tax
GVA	Gross Value Added
HTTP	Hypertext Transfer Protocol
IIP	Index of Industrial Production
IMF	International Monetary Fund
JSON	JavaScript Object Notation
KG	Knowledge Graph
LLM	Large Language Model
LPA	Long Period Average (rainfall)
MCP	Model Context Protocol
MoSPI	Ministry of Statistics and Programme Implementation
MPC	Monetary Policy Committee
NPA	Non-Performing Asset
PLFS	Periodic Labour Force Survey
PMI	Purchasing Managers' Index
RAG	Retrieval-Augmented Generation
RBI	Reserve Bank of India
SDF, MSF	Standing Deposit Facility, Marginal Standing Facility
SQL	Structured Query Language
SSE	Server-Sent Events
UI	User Interface
VAR	Vector Autoregression
WALR	Weighted Average Lending Rate
WPI	Wholesale Price Index
YoY	Year on Year

Chapter 1
Introduction

1.1 INTRODUCTION

Monetary policy, Fiscal planning, Investment strategy, and corporate decision making are all dependent on macroeconomic analysis. A policy analyst interested in the impact of an increase in crude oil prices on the Indian economy would need to consult data on inflation published by the Ministry of Statistics and Programme Implementation (MoSPI), data on the policy rates and credit by the Reserve Bank of India (RBI), fiscal data by the Controller General of Accounts, trade data by the Directorate General of Commercial Intelligence and Statistics, and data on the markets by the exchanges. Every agency releases in different kinds of formats, at varying intervals and with different time periods. Analysis is, therefore, a long series of data collection, cleaning, aligning and cross-sector reasoning prior to a single sentence of interpretation.

The large language models (LLMs) that can create fluent economic narratives on demand are a new phenomenon, but fluency does not equal correctness. If the model is queried for the latest reading of the Consumer Price Index (CPI) it can return a believable but non-existent number. This is a common pattern that has been widely observed (Ji et al. 2023) and is particularly harmful in economics, where a single incorrect figure can alter the outcome of a conclusion. The problem is reduced by playing the text that is retrieved into the model, but the model still writes the numbers and there is no guarantee that it copies them accurately (Lewis et al., 2020).

Macrograph AI is the solution to the problem from the system side. It breaks down the Indian economy into agents of the ten sectors, with each sector possessing a set of indicators and having its own data-access server from which it receives the indicators from official sources. The agents communicate with each other using a typed agent-to-agent protocol, the protocol is orchestrated by a LangGraph, a causal knowledge graph ties the indicators together, and each number in the final report has a provenance hash, observation period and source. If the source is not available, the system does not attempt to fill the gap, but rather reports that the gap is unavailable.

1.2 OVERVIEW OF MACROECONOMIC INTELLIGENCE SYSTEMS

A macroeconomic intelligence system is a software system that gathers macroeconomic observations, correlates these observations and displays conclusions to a human decision maker. Traditional solutions include dashboards and econometric toolkits. A dashboard shows information series for one or multiple sources, and allows for interpretation by the dashboard user. A toolbox of econometric models can be estimated using an econometric model, for example a vector autoregression, but the analyst has to prepare the data and have to define the model. None respond to an open-ended question like the impact of a crude oil shock on inflation and the policy rate. There have been recent attempts to use LLM-based agents for economic and financial applications. Some simulate a heterogeneous household and its policy response (Hao & Xie, 2025), some simulate the survey respondents as macroeconomic expectations (Lin et al. 2026), and some simulate a trading firm with analyst, researcher and trader agents (Xiao et al. 2025). Another area of related research is to make use of a knowledge graph of economic variables for retrieval and forecasting (Yang et al., 2020; Zhu et al., 2025), and a recent approach is to create an economic story from actual model computation (Kato, 2026). These contributions are discussed in detail in Chapter 2. Such systems can be built from independent components due to two open protocols. The Model Context Protocol (MCP) is a standard that simplifies calling tools and reading data from models (Anthropic, 2024). The Agent2Agent (A2A) standard is a new standard for agent discovery and task exchange that has been developed to reduce the complexity of finding and interacting with agents (Google, 2025). Macrograph AI utilizes MCP for data access, and a typed A2A layer for agent cooperation - both are kept completely separate: reasoning flows between agents, data is only shared from the owner of the data.

1.3 CHALLENGES PRESENT IN AUTOMATED MACROECONOMIC ANALYSIS

The design of this project was influenced by the several technical and methodological challenges in building a trustworthy automated analyst for the Indian economy.

• Sources that are diverse and not consistent. Many agencies publish their official indicators as APIs, spreadsheets, PDF bulletins and web tables, with varying base years, units and release schedules.

• Unsupported numbers. An LLM can quote a number which is not found in any official announcement. It is difficult to detect after they have happened, as the figure seems plausible.

• Data that has been copied or is duplicated. Multiple agents may retrieve the same indicator independently and get conflicting results or the report might be self-contradictory.

• Implicit causality. Macroeconomic variables have opposite signs and respond to each other with lags. A system that does not take these transmission channels into account can describe any individual series but not explain a shock.

• Weak provenance. A reader will have to be able to determine where a number is from, which table, which period and when it is retrieved. This information is seldom found in narrative text.

Partial outages of sources. The government endpoints have slow or inconsistent service availability. A system should fail gracefully, notify the user of missing components and do not create a replacement.

• Honest uncertainty. Any results or relationships assumed in the simulation must be marked as such. A heuristic projection is presented as a measured fact misleading the reader.

1.4 PROJECT STATEMENT

To design, implement and evaluate a multi-agent platform for Indian macroeconomic intelligence where: (i) specialised agents in each sector own their indicators and fetch them from official sources, (ii) agents cooperate with other agents following a typed and auditable protocol, which separates reasoning from data access, (iii) a causal knowledge graph and a transparent scenario engine connect indicators across different sectors, and (iv) every claim in the generated report is referable to a source, an observation period and a provenance record, with missing data reported as unavailable.

1.5 OBJECTIVES

The project aimed at the following objectives; Figure 1.1 shows the relationship of the problems to their respective objectives and to the component that addresses the problem.

1. O1. Sector-wise data access. Give one set of data-access server and one set of local stores for 10 macroeconomic sectors supported by official sources.
2. O2. Coordinated multi-agent reasoning. Propose a typed protocol for agents to communicate with each other that allows for discovery, routing and loop prevention, and tracing of the communication, and an orchestrator to break down the question and combine the answers.
3. O3. Source-grounded answers. Implement source provenance for successful responses at protocol level and return structured unavailable states when there are no estimates.
4. O4. Explicit causal model. Construct knowledge graph of cross-sectoral transmission channels with lags, sign and evidence class; explain and simulate shocks using knowledge graph.
5. O5. Traceable provenance. Attach source, table, period, retrieval time and a hash to every observation and record the delegation tree of every conversation.
6. O6. Research instruments. Offer tools for testing the model against evidence (Edge evidence audit, Anomaly radar), exploring it interactively (Trajectory and Twin views) and verifying claims against evidence.

7. O7. Usable interface. Provide a web interface with a cited dashboard, streaming chat, causal lab and agent registry.

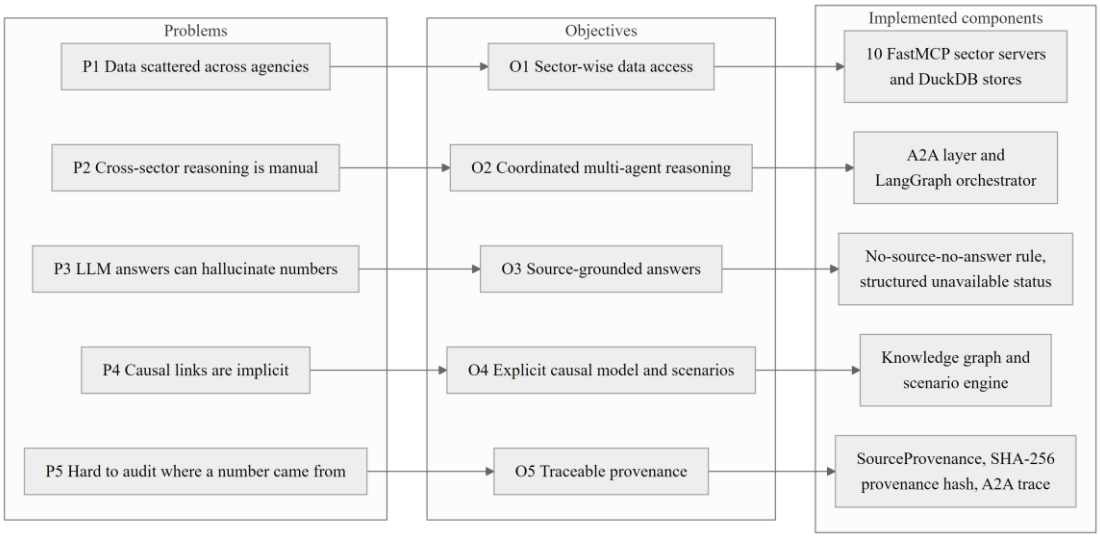


Figure 1.1: Mapping of identified problems to project objectives and implemented components.

1.6 SCOPE OF THE PROJECT

The platform is divided into 10 sectors of the Indian economy: real sector, agriculture, prices and inflation, monetary conditions, finance and banking, capital markets, fiscal affairs, labour and employment, external sector, and services. Each sector is a python package that contains a data-access server, a local DuckDB store, a Python package containing an agent and an interface for the agent-to-agent protocol. The system is completed by the orchestrator, the knowledge graph, the scenario engine, the research lab, the daily brief and the web application. Figure 1.2 is a differentiation between what is implemented, what is prototype and what is out of scope.

The scope is intentionally limited. The platform is not an investment advisor and does not make investment forecasts with statistics and does not make causal identification. Retrieval of documents (for example RBI minutes) is not in place, thus all reports indicate that there is a lack of documentary evidence. The claim verification library is implemented and tested but not yet in the answer path.

1.7 CONTRIBUTIONS OF THE WORK

- A ten-agent architecture with a protocol level constraint on ownership of the indicators: only seven sector to sector exchanges are declared and all others are refused.
- A native typed agent-to-agent layer with guards for depth, hops, cycles and duplicates, along with an MCP-based data layer with a live, cached, unavailable fallback chain.
- A causal knowledge graph which features a five-tier taxonomy of evidence and a clear graph-attenuation scenario engine with clearly communicated and labeled assumptions.
- A data-informed edge evidence audit that helps distinguish seeded priors from relationships that are supported by data, as well as an anomaly radar that helps classify outliers as to whether the connected indicators moved in the same way.
- A claim verification library which recalculates numerical claims with precise decimal arithmetic, using content addressed evidence, without ever claiming model prose as fact.
- A web application containing a cited dashboard, streaming chat, A2A registry and an interactive causal lab, supported by an automated test suite of 455 test functions (551 executed cases).

1.8 ORGANISATION OF THE REPORT

The report is organised into ten chapters. Table 1.1 summarises the content of each chapter. The work was carried out between July and October 2026, in the phases listed in Table 1.2.

Table 1.1: Organisation of the report

Chapter	Title	Content
1	Introduction	Motivation, challenges, problem statement, objectives, scope, contributions.
2	Background	Survey of LLM agents, retrieval, knowledge graphs, protocols, causality and verification; research gaps.
3	System Requirements and Design	Requirements, architecture, data model and design decisions.
4	Protocol Design	Agent-to-agent protocol, MCP data layer, routing, loop protection and provenance.
5	Implementation	Orchestrator, sector agents, data acquisition and resilience.
6	Knowledge Graph and Scenario Engine	Ontology, graph queries, scenario algorithm and worked examples.
7	Research Lab and Verification	Trajectory, twin, anomaly radar, MPC, edge audit, daily brief, claim verification.
8	User Interface and API	Web application, streaming protocol and HTTP interface with screenshots.
9	Testing, Results and Discussion	Test strategy, measured results, evaluation plan and limitations.
10	Conclusion and Future Work	Summary, contributions and roadmap.

Chapter 2
Background

2.1 INTRODUCTION

This chapter examines the literature and technologies Macrograph AI is built upon. It provides an overview of agents with LLM capabilities for economics and finance, retrieval-augmented generation in the financial space, knowledge graphs for macroeconomics, protocols for interoperability between agents, causal and scenario analysis, and research on trustworthy generation. The chapter ends with a comparative summary and the research gaps that are motivating the project and the technologies selected. During the project, seven reference papers have been gathered, which are classified into five themes: LLM agents in economics and finance, retrieval for financial question answering, knowledge graphs, interoperability protocols and trustworthy generation.

2.2 SURVEY ON LLM-BASED AGENTS FOR ECONOMICS AND FINANCE

2.2.1 AGENT-BASED MODELLING WITH LANGUAGE MODELS

In classical terms, an agent is a computer system that resides in an environment that it can perceive and that can act independently to achieve its design goals (Wooldridge & Jennings, 1995). Interest in agents increased due to the emergence of Large Language Models which can plan, call tools and communicate in natural language. Generative agents, consisting of a language model, memory, and reflection, have been demonstrated to generate coherent individual and social behavior (Park et al., 2023), and the ReAct pattern fuses reasoning traces with tool actions (Yao et al., 2023). These ideas form the basis of the sector agents and the orchestrator for this project.

2.2.2 ECONOMIC AND POLICY ANALYSIS WITH MULTIPLE LANGUAGE MODELS

Hao and Xie (2025) suggest an MLAB framework to analyze economic and public policies. They first compare the performance of five LLMs for two-period consumption allocation problems with and without an explicitly given utility function, and then compare the models across educational groups and income brackets to replace heterogeneity of households with differences in analytical ability. The case study is interest-income taxation. The work demonstrates that LLM

agents can play the role of diverse economic agents but the agents solve stylised decision tasks and do not access the observed macroeconomic data. Lin et al. (2026) present economic agents based on large language models that are able to mimic expectations in an experimental setting. Every agent has modules that pull personal features, previous expectations and dynamic external information. The framework is checked by reproducing three exemplary survey designs, and the authors find that the generated expectation distributions are very close to human data, and priors play a key role in reproducing distributions. This work highlights the importance of the structured retrieval modules that are implemented around the model, such as the sector agents in Macrograph AI.

2.2.3 MULTI-AGENT FINANCIAL TRADING

TradingAgents (Xiao et al., 2025) is an organization of language-model agents as a trading company, including fundamental analysts, sentiment analysts, technical analysts, Bull and Bear researchers that discuss market conditions, risk management team, and traders with various risk preferences. The authors claim that their models outperform baseline models in terms of cumulative returns, Sharpe ratio and maximum drawdown. The framework provides a good example of how role specialisation and the structured debate is beneficial, but there is no ownership of a data point assigned to one of the agents, and the agents share a common data feed.

2.2.4 THE AI ECONOMIST

Zheng et al. (2022) design tax policy with a two-level deep multi-agent reinforcement learning. It solves a different problem: optimal policy design within a simulated economy, but it is a milestone in the way multi-agent methods are applied in the field of economics. A similar name is used in a different system by Kato (2026) as described in 2.4.

2.3 RETRIEVAL-AUGMENTED GENERATION IN FINANCE

Retrieval-augmented generation (RAG) is a combination of a parametric language model and a memory component that is searched during generation (Lewis et al., 2020). Retrieved passages are included in the prompt for the model to use as context for answering the prompt. As of now, the most commonly used approach for reducing hallucination in knowledge-intensive tasks is RAG, albeit there are numerous ways to chunk documents, index and re-rank them, each with a different

quality. The question is studied by Dadopoulos et al. (2025) in the context of financial statements. Multi-stage metadata-driven RAG architecture is evaluated and benchmarked on the FinanceBench dataset, which includes pre-retrieval filtering, post-retrieval re-ranking and enriched embeddings. They discover that a powerful re-ranker is necessary for accuracy, they realize that the greatest gains are derived from including chunk metadata with the text (contextual chunks) and they determine that a custom metadata re-ranker works well for them at more affordable price tag than commercial re-ranker. The study is relevant here on two grounds. First, it proves that the quality of retrieval in the finance domain is dependent on metadata, in the context of Macrograph AI, this translates to the source, table, period and unit. Second, in this project, it is the macroeconomic questions which are mostly answered by numerical series, while questions for documents are targeted. Therefore, in this project structured observations are retrieved through tools.

2.4 KNOWLEDGE GRAPHS FOR MACROECONOMICS

A knowledge graph is a directed labelled graph that expresses entities and typed relationships as directed labelled graph (Hogan et al., 2021). In economics, a node may be a variable (or a set of variables) and an edge may state that one variable affects another. Using natural language processing techniques to analyze academic literature and research reports, Yang et al. (2020) build a knowledge graph of macroeconomic variables based on that information, including alternative big data variables. They leverage the graph as prior knowledge for variable selection in forecasting and report much greater forecasting accuracy than statistical variable selection, particularly for long-run forecasts. Macrograph AI's graph is a much smaller, hand-curated version of the same purpose, to encode prior knowledge about transmission channels. Zhu et al. (2025) have proposed a Knowledge Graph-Guided RAG (KG2RAG) framework. The method expands seed chunks from the knowledge graph, and organizes the expanded chunks into coherent paragraphs after a semantic retrieval step to provide seed chunks. HotpotQA and its variants demonstrate improvements in response quality and retrieval quality with experiments. The result confirms the notion that graph structure provides fact-level relationships which are missing in isolated chunks. The most similar one is the AI economist agent of Kato (2026). It is an agentic, RAG-based model grounded approach that leverages a knowledge graph with typed nodes for text evidence,

model specifications and model runs. The language model does not create quantitative claims, but rather model computations are run and quantitative claims are written in the graph. The framework is tested across two applications: (1) U.S. inflation persistence with Fed policy; (2) commercial real estate refinancing stress; and compared across a variety of scenarios: LLM-only; RAG-only; model-grounded GraphRAG. The central finding is qualitative, asking the generator to provide evidence retrieved from a text to support their writing, to sketch paths and to show the model output performed to justify their finding. The idea behind Macrograph AI is the same as with the numbers: numbers should never come from the language model, but it works differently. It benchmarks numbers against official observations in the sector, establishes provenance in a multi-agent protocol, classifies evidence class for each causal edge and focuses on the Indian economy.

2.5 AGENT INTEROPERABILITY PROTOCOLS

Anthropic (2024) proposed a new client-server standard called Model Context Protocol (MCP) that enables applications to present data and tools to language-model clients. The parameters of the tool definition are all typed, so an agent may learn about and invoke any of them in a uniform manner. Every sector in Macrograph AI provides access to its data via a FastMCP server.

Google (2025) is tackling a different issue with the Agent2Agent (A2A) protocol: how agents developed by different teams identify their peers' capabilities and share responsibilities. It adds support for agent cards which promote skills and a task-focused message exchange. Macrograph AI builds on these ideas with layer types, with loop protection guards, with source provenance and with a declared dependency graph, and with keeping A2A strictly for reasoning, and MCP strictly for data. The two channels are illustrated in Figure 2.1.

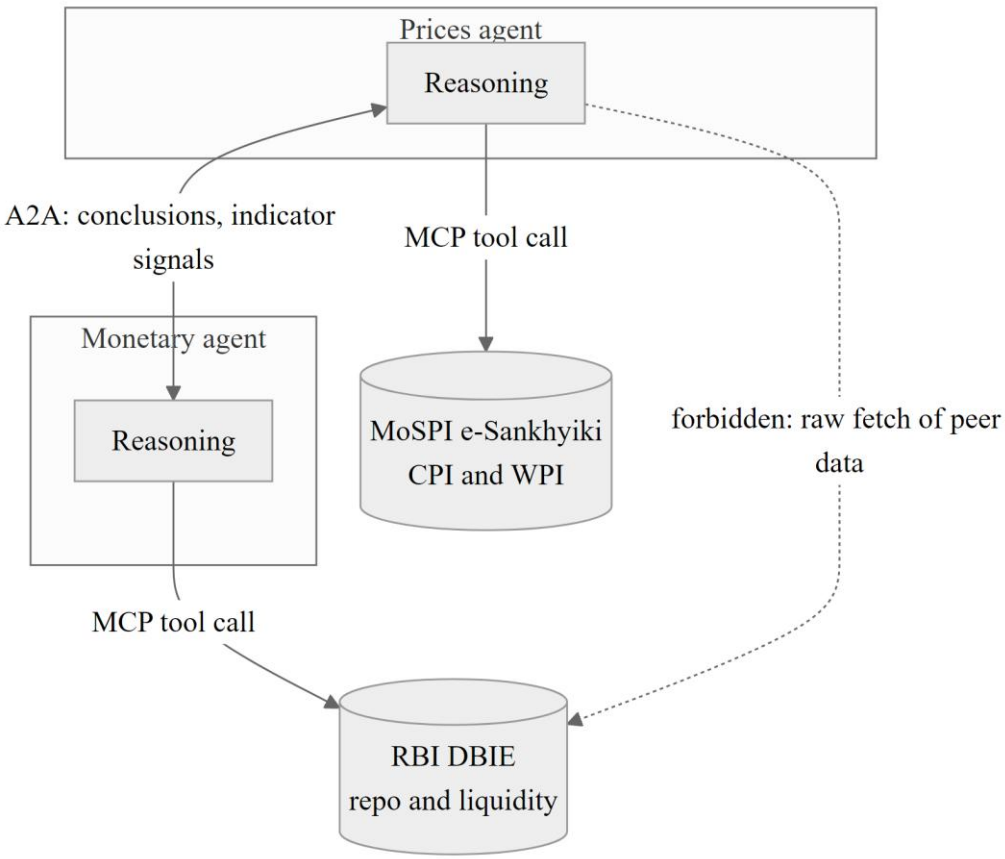


Figure 2.1: Separation of the agent-to-agent reasoning channel (A2A) and the agent-to-data channel (MCP).

2.6 CAUSALITY AND SCENARIO ANALYSIS IN MACROECONOMICS

Macroeconomists are able to tell the difference between association and causation. Vector autoregressions were developed by Sims (1980) for studying the impact of shocks while avoiding the requirement of specifying a complete structural model. Sims (1980) provided an introduction to vector autoregressions for studying dynamic effects of shocks without the requirement of a full structural model. Jordà (2005) suggested use of local projections to estimate impulse responses from horizon to horizon and be robust to misspecification of the dynamic system. Pearl (2009) formalized the structural causal model and conditions for inferring causal effects from data. Care must also be taken with time-series tests: When a unit root exists in the series, the series must be differenced (Dickey & Fuller, 1979) and testing many relationships must control the false discovery rate (Benjamini & Hochberg, 1995).

These references clarify what the Macrograph AI scenario engine is and what it isn't. It has a propagation rule that is a simple heuristic that does not attempt to estimate

impulse responses, and it reports the results as simulated, and tests for edges based on a Granger test to ensure the edges are supported by the available data. The main item of future work is to replace the heuristic with local projections.

2.7 TRUSTWORTHY GENERATION AND HALLUCINATION

Ji et al. (2023) conducted a survey of Hallucination in NLPG and separated it into contradiction with source and inabilitng to verify with source. In the case of economic text, the practical outcome is that a figure created needs to be compared to the evidence retrieved. There are three families of mitigation that can be found in the literature: grounding generation in retrieved text, limiting the model to call tools for facts, and verifying generated claims after generation. Macrograph AI integrates the first two, and adds the third as a claim verification library to reprocess numerical claims and only accept qualitative claims as exact source sentences.

2.8 COMPARATIVE SUMMARY OF RELATED WORK

Table 2.1 compares the reviewed systems along the dimensions that matter for this project.

Table 2.1: Comparison of related work and Macrograph AI

System	Approach	How numbers are grounded	Provenance and verification	Domain
Hao & Xie (2025)	Heterogeneous LLM households for policy analysis	Simulated decision problems	Not addressed	Public policy
Lin et al. (2026)	Macroeconomic expectations in survey experiments	Retrieved priors and external information	Validated against human survey data	Survey experiments
Xiao et al. (2025)	LLM trading firm	Shared market and news data feeds	Backtest metrics; no per-claim verification	Equity trading
Yang et al. (2020)	Knowledge graph for variable selection	Graph extracted from literature by NLP	Forecast accuracy evaluation	Macro forecasting
Zhu et al. (2025)	KG-guided RAG	Retrieved chunks expanded by graph	Retrieval and response quality metrics	Open-domain QA
Dadopoulos et al. (2025)	Metadata-driven RAG	Contextual document chunks	FinanceBench accuracy	Corporate filings
Kato (2026)	Model-grounded GraphRAG economist	Executed model computations in the graph	Traceable evidence and model-run nodes	U.S. inflation and bank stress
Macrograph AI (this work)	Ownership-constrained multi-agent platform	Sector-owned official observations through MCP	Protocol-level provenance, edge evidence audit, claim verifier (library)	Indian macroeconomy

2.9 RESEARCH GAPS AND MOTIVATION

Role specialised agents, knowledge graphs and grounded generation are each well-studied, as shown in the review. There are four sections of blank space to be filled with a platform that should be able to provide cross-sector answers to specific questions related to an economy from official statistics.

1. **Data ownership across agents.** Typically, multi agent systems have one data feed. A rule that ensures that each indicator has exactly one owner, only other agents can gain access to it in a declared exchange does not preclude duplicate and conflicting values, but is not often stated or enforced.
2. **Evidence grades for causal edges.** Normally, knowledge graphs are represented as facts. It distinguishes typed-in by designer relationships from supported relationships (via data), and makes it apparent to the user, which helps with honest scenario analysis.
3. **Machine-checked numbers.** Grounding is the process used to reduce hallucination, but it does not indicate the correct answer. This is addressed by the deterministic recomputation of claims with evidence.
4. **Coverage of India.** Focused published systems are based in the USA and equity markets. There is need for an Indian platform to incorporate MoSPI, RBI and the peculiarities of their dispositions.

2.10 TECHNOLOGIES USED

Table 2.2 lists the technologies used in the implementation and the role of each. PydanticAI and a vector database appeared in early design documents but are not part of the implemented system.

Table 2.2: Technologies used in Macrograph AI

Layer	Technology	Role
Language and runtime	Python 3.12, uv	Backend implementation and dependency management
Orchestration	LangGraph (LangChain, n.d.)	Stateful four-node orchestration graph
Data tools	FastMCP (Lowin, n.d.)	One MCP tool server per sector
API gateway	FastAPI (Ramírez, n.d.), Uvicorn	HTTP and Server-Sent Events interface
Models and validation	Pydantic v2 (Pydantic Services Inc., n.d.)	Typed messages, observations and claims
Analytical store	DuckDB (Raasveldt & Mühleisen, 2019), SQLite	Sector stores, canonical store, claim audits
Graph and statistics	NetworkX (Hagberg et al., 2008), statsmodels (Seabold & Perktold, 2010), pandas	Knowledge graph engine, Granger tests, series handling
Language models	Groq-hosted open-weight models through langchain-groq; offline fallback	Narrative synthesis and routing; deterministic mode without a key
Frontend	React 18, TypeScript, Vite, Tailwind CSS, GSAP, Mermaid	Dashboard, chat, causal lab and registry views

Chapter 3
System Requirements and Design

3.1 INTRODUCTION

This chapter defines Macrograph AI requirements and outlines their design: System context, Layered architecture, Structure of a sector agent, Ownership model for indicators and Data model. More information on protocol details is in Chapter 4 and implementation details is in Chapter 5.

3.2 REQUIREMENTS ANALYSIS

3.2.1 FUNCTIONAL REQUIREMENTS

The functional requirements were derived from the objectives in Chapter 1 and from the behaviour expected by an analyst. They are listed in Table 3.1; the non-functional requirements are in Table 3.2.

Table 3.1: Functional requirements

ID	Requirement	Objective
FR1	Each of the ten sectors shall expose its indicators through its own MCP server.	O1
FR2	Each sector shall keep a local DuckDB store and tag every value as live, cached or unavailable.	O1, O3
FR3	A user shall be able to ask a question to one sector agent directly or to the orchestrator.	O2, O7
FR4	The orchestrator shall route a question to relevant agents using their Agent Cards.	O2
FR5	Agents shall exchange requests and responses as typed messages with correlation identifiers.	O2
FR6	The protocol shall reject delegation loops, over-deep chains, duplicates and undeclared exchanges.	O2
FR7	A successful response shall carry source provenance; failures shall return structured errors.	O3, O5
FR8	The system shall map transmission paths between indicators in a knowledge graph.	O4
FR9	The system shall simulate a user-specified shock and report its labelled results.	O4
FR10	The report shall list observations with period, status and provenance hash, and name unavailable agents.	O3, O5
FR11	The system shall test graph edges against data and detect anomalies in available series.	O6
FR12	The web application shall provide a dashboard, streaming chat, causal lab and registry view.	O7

3.2.2 NON-FUNCTIONAL REQUIREMENTS

Table 3.2: Non-functional requirements

ID	Requirement	How it is addressed
NFR1	Traceability: every figure can be traced to a source.	SourceProvenance on every response; provenance hash on observations.
NFR2	Resilience: partial failure must not abort the run.	Structured errors, partial status, cached fallback, retries with exponential backoff.
NFR3	Modularity: sectors can be added without changing others.	Registry bootstrap is the only module naming sectors; no sector imports another.
NFR4	Safety: loops and runaway delegation must be impossible.	Depth 5, 25 requests per conversation, cycle and duplicate detection.
NFR5	Security of data access: no SQL injection.	Parameterised DuckDB queries; typed Pydantic validation.
NFR6	Availability without credentials.	Deterministic offline synthesis when no language model key is configured.
NFR7	Maintainability.	Pydantic v2 models, a 400-line module guideline (exceptions are listed in Chapter 9), 455 automated test functions.
NFR8	Honest labelling.	SIM, CACHED, heuristic, SEEDED and EMPIRICAL labels in payloads and interface.

3.3 SYSTEM CONTEXT

Figure 3.1 is the system boundary. The Webapp is used by analysts and researchers, and the HTTP interface can be called directly by the developers. The platform reads official sources through MCP servers and web interfaces, calls a language model provider for narrative text, optionally mirrors the knowledge graph to Neo4j and uses Google Identity Services for sign-in.

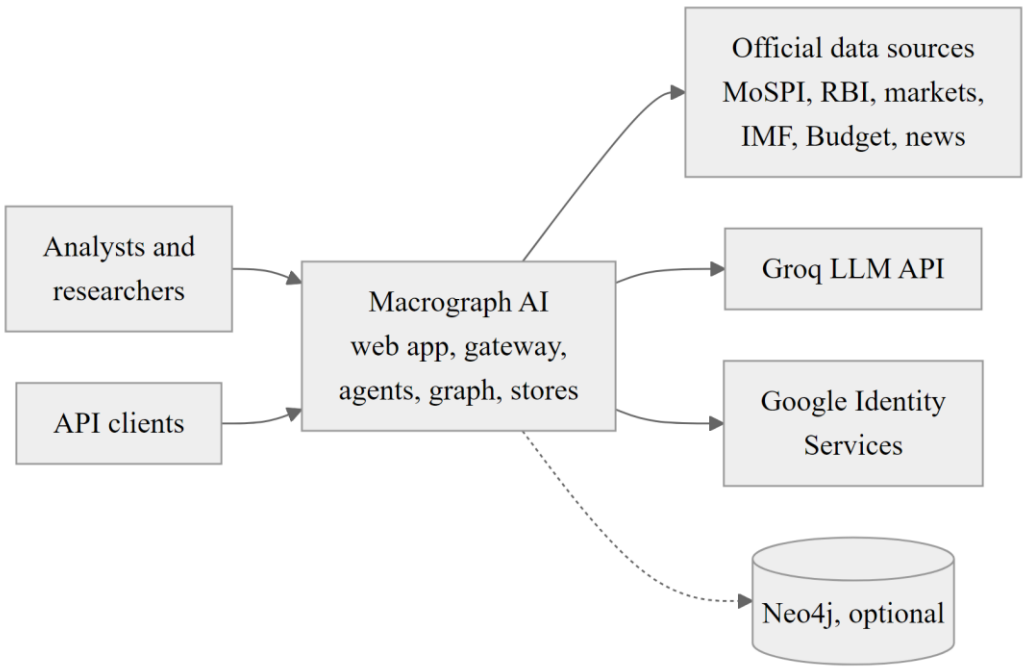


Figure 3.1: System context of Macrograph AI: users, external data sources and external services.

3.4 LAYERED ARCHITECTURE

The system comprises of 7 layers as illustrated in Figure 3.2. Calls are only allowed to go down. This is a react based presentation layer. The gateway layer is a FastAPI application, with the aim of exposing the following: the chat and dashboard routers, the research router, the analysis and simulation endpoints, the a2a interface, the mounted sector sub-applications. Orchestration layer is a LangGraph state graph. There are 10 sector agents in the agent layer. The protocol layer is for the A2A server and client, and the MCP servers and client. The knowledge and analysis layer contains the ontology as well as the graph engine, the scenario engine, the research lab and the claim verification library. The data layer interacts with the official sources and the language model API, as well as stores the DuckDB and SQLite stores.

In the development configuration, all the agents are contained in a single Uvicorn process, which means the A2A transport is in-process. The Vite development server makes API calls to the gateway on port 8000. DuckDB files are local and Neo4j is optional and disabled if no URI is provided.

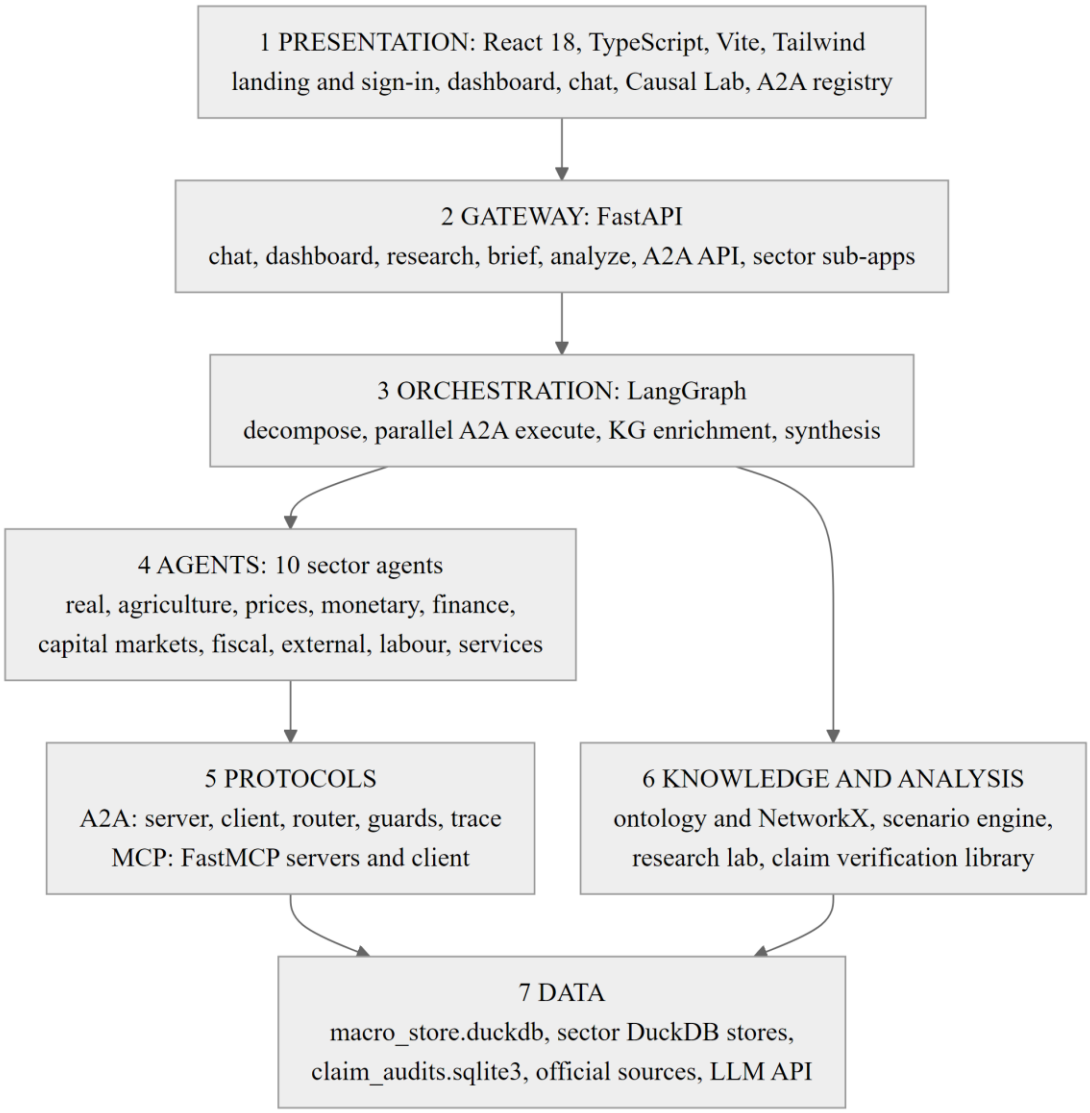


Figure 3.2: Layered architecture of Macrograph AI.

3.5 DESIGN OF A SECTOR AGENT

All sector packages are organized in the same way, as illustrated in Figure 3.3, with the exception of their maturity. The client module makes asynchronous HTTP calls and retries with an exponential backoff, the parsers normalise the payloads, the Pydantic models validate the payloads, the database module defines the DuckDB schema and caches results with parameterised SQL, the MCP server exposes typed tools, the agent node reasons over the data, the executor exposes the agent to the A2A server, and the sub-application exposes HTTP routes.

A single-sector question can bypass the orchestrator. Each agent module provides a check of the form *is_direct_<sector>_query*, and the shared helper *core/sector_reasoning.py* provides evidence-grounded reasoning for direct queries.

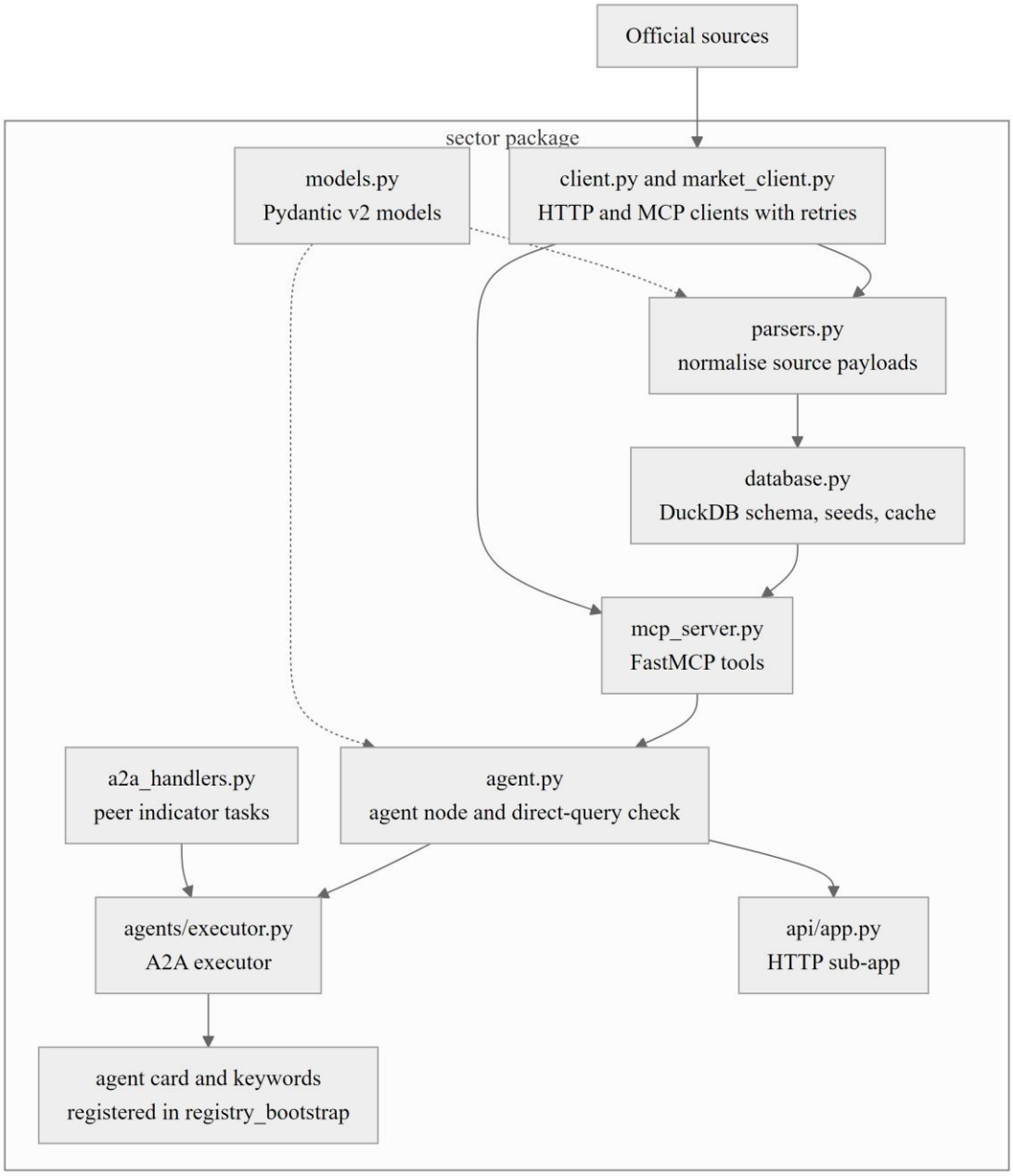


Figure 3.3: Standard internal structure of a sector agent package.

3.6 INDICATOR OWNERSHIP AND DEPENDENCIES

The central design rule is single ownership: every indicator has exactly one owner agent, for example CPI and WPI belong to the prices agent and the repo rate belongs to the monetary agent. Another agent that needs an owned indicator must obtain it from the owner through an A2A exchange and must never recompute or re-fetch it. Only seven sector-to-sector exchanges are declared (Figure 3.4); the orchestrator may call every agent, and all other pairs are refused with the error code *DEPENDENCY_NOT_ALLOWED*.

Table 3.3 lists the owner of each group of indicators.

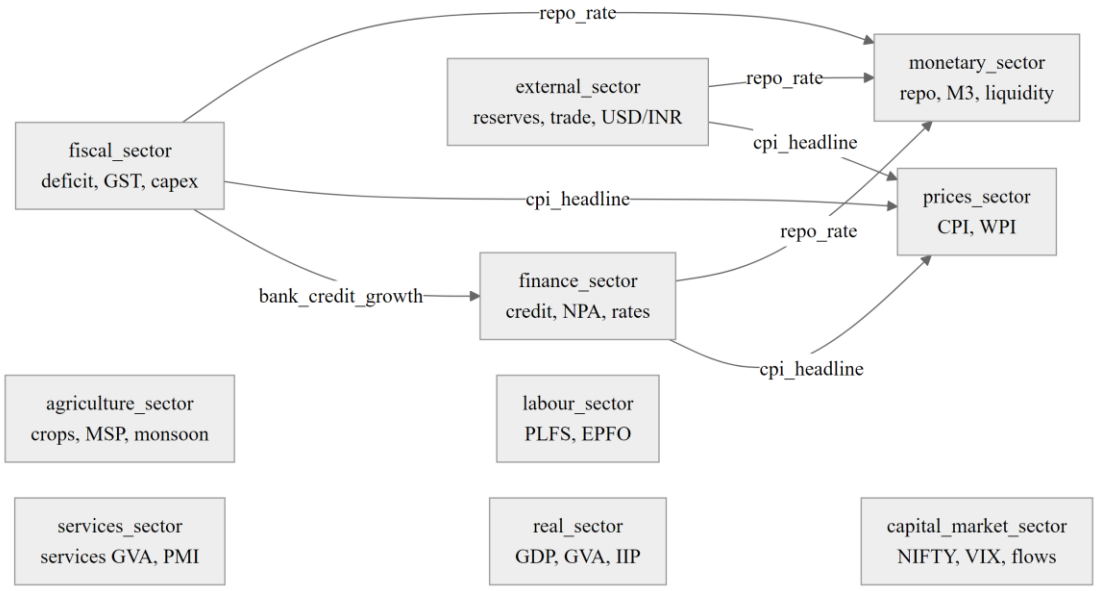


Figure 3.4: Indicator ownership and the seven declared sector-to-sector A2A dependencies.

Table 3.3: Indicator ownership by sector agent

Owner agent	Primary indicators
prices_sector	CPI, WPI, food, fuel and core inflation
monetary_sector	Repo rate, SDF, MSF, M1 and M3, liquidity
finance_sector	Bank credit growth, NPA ratios, lending and deposit rates
fiscal_sector	Fiscal deficit, GST collections, Union capital expenditure
external_sector	Trade balance, forex reserves, balance of payments, USD/INR
real_sector	GDP, GVA, IIP, gross fixed capital formation
capital_market_sector	NIFTY indices, India VIX, FII and DII flows, G-Sec yields
agriculture_sector	Crop production, MSP, mandi prices, monsoon rainfall
labour_sector	PLFS unemployment, labour force participation, EPFO payroll additions
services_sector	Services GVA, services PMI, IT and ITeS exports

3.7 DATA MODEL

The four main entities are presented in Figure 3.5. A CanonicalIndicator is a description of what can be measured, e.g. in.macro.prices.cpi_headline, a unit, a frequency and a source authority. A CanonicalObservation is one measured value with its period, retrieval timestamp, revision status, data status and a provenance hash. Two indicators are connected with a type, lag, sign, p-value field, and confidence and mechanism text by a CausalRelationship. A ScenarioResult consists of a simulated shock and the identifiers of the relationships that carried a shock. Policy rates, money supply, system liquidity and monetary stance tables are kept separately for each sector as well, for instance in monetary store, while every store has its own fetch_log table.

The daily briefs and a news cache are also stored by the canonical store macro_store.duckdb.

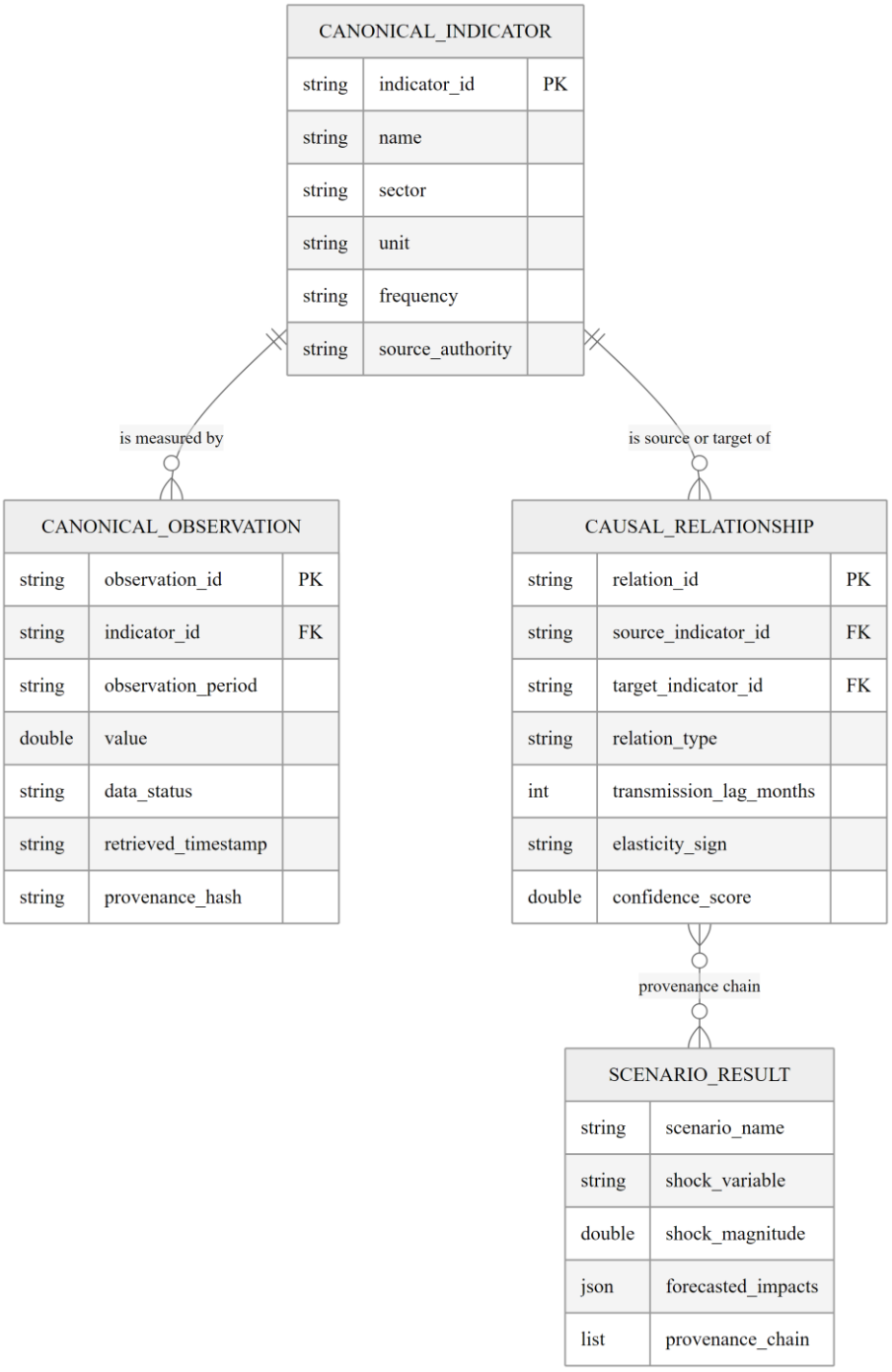


Figure 3.5: Entity-relationship diagram of the canonical economic data layer.

3.8 DESIGN DECISIONS AND RATIONALE

Table 3.4 records the principal design decisions and the reason for each.

Table 3.4: Principal design decisions and their rationale

Decision	Rationale
Native typed A2A layer instead of a library	The project needed guards, mandatory provenance and a declared dependency graph, which are easier to enforce in one server.
One MCP server per sector	Keeps data access inside the owner and removes the need for cross-sector data calls.
In-process transport by default	All agents share one gateway; an HTTP transport can replace it without changing callers.
DuckDB for stores	Embedded analytical SQL with no server to operate; parameterised queries.
NetworkX for the graph, Neo4j optional	In-memory queries are sufficient for twenty nodes; Neo4j mirrors the ontology for persistence.
Deterministic fallbacks everywhere	Allows demonstration, testing and operation without credentials.
Heuristic scenario engine labelled SIM	Transparent and explainable; replaced later by estimated models (Chapter 10).
Structured unavailable status	Prevents silent substitution of invented values.

Chapter 4
Protocol Design

4.1 INTRODUCTION

Macrograph AI uses two communication channels with strictly different purposes. The agent-to-agent (A2A) channel carries reasoning: delegated tasks, structured conclusions and indicator signals. The MCP channel carries data: typed tool calls that read official sources and local stores. An agent never fetches another agent's data; it obtains a peer's indicator only by asking the peer over A2A. This chapter describes the A2A layer, the MCP data layer and the way provenance is preserved across both.

4.2 THE AGENT-TO-AGENT LAYER

4.2.1 MESSAGES

The A2A layer lives in *backend/core/protocols/a2a*. All messages are strict Pydantic v2 models that forbid extra fields. An *A2ARequest* carries a request identifier, sender, receiver, task, parameters, conversation identifier, the identifier of its parent request, the current delegation depth and the chain of agents that have already been called. An *A2AResponse* carries the status (success, partial or failed), a result, a list of *SourceProvenance* records, structured errors, optional follow-up requests and metadata.

Table 4.1 lists the fields of the principal messages.

Table 4.1: Fields of the principal A2A messages

Message	Fields
A2ARequest	request_id, sender_agent, receiver_agent, task, parameters, context, conversation_id, parent_request_id, depth, call_chain, timeout_seconds, timestamp
A2AResponse	request_id, sender_agent, receiver_agent, status, result, sources, errors, follow_ups, metadata, timestamp
SourceProvenance	source_name, source_url, dataset, table, reporting_period, retrieved_at, page_or_section, record_reference
A2AErrorInfo	code, message, agent
IndicatorSignal	owner_agent, observations (indicator_id, label, value, unit, observation_period, data_status)

4.2.2 THE A2A SERVER

All requests pass through one *A2AServer*. Its *handle* method applies a fixed pipeline (Figure 4.1). The receiver must be registered; the loop guard must pass; the sender

must either be unrestricted (the orchestrator and the gateway) or hold a declared dependency on the receiver; the task must be in the receiver's supported tasks and have a registered handler; the parameters must match the task's parameter model. The handler then runs under a timeout, and the result must have a valid type and, unless the response failed, at least one source. A violation at any step produces a failed response with a stable error code. No exception crosses from one agent to another.

The rule that a non-failed response must carry sources enforces the principle “No Source, No Answer” at protocol level. A handler cannot return a bare answer; it must return a *HandlerOutput* whose sources the server preserves unchanged. Table 4.2 lists the error codes.

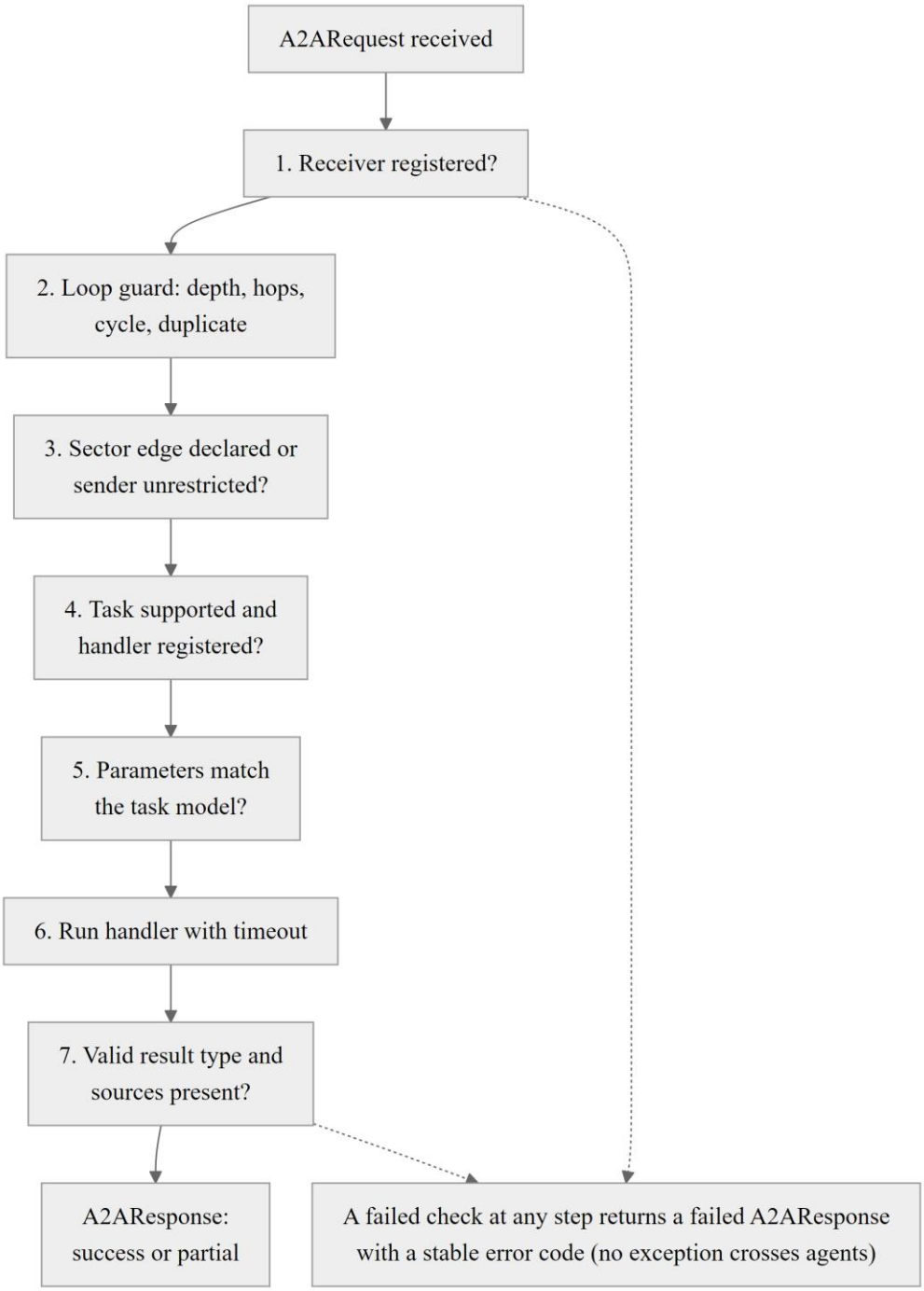


Figure 4.1: Validation pipeline applied by the A2A server to every request.

Table 4.2: A2A error codes

Code	Meaning
INVALID_REQUEST	Bad parameters or malformed call chain
AGENT_NOT_FOUND, AGENT_UNAVAILABLE	Unknown agent, or an advertised task without a handler
UNSUPPORTED_CAPABILITY	Task not listed in the receiver's supported tasks
DEPENDENCY_NOT_ALLOWED	Sector-to-sector edge not declared
MAX_DEPTH_EXCEEDED, MAX_HOPS_EXCEEDED	Delegation too deep, or too many requests in a conversation
CIRCULAR_DELEGATION, DUPLICATE_REQUEST	Receiver already on the call chain, or an identical request repeated
AGENT_TIMEOUT	Handler exceeded its timeout (retryable)
DATA_SOURCE_ERROR, LLM_ERROR, AGENT_ERROR	MCP or upstream failure, language model failure, any other handler exception
INVALID_RESPONSE	Schema violation, missing sources, wrong type or correlation mismatch
TRANSPORT_ERROR	Delivery failed (retryable)

4.2.3 LOOP PROTECTION AND RELIABILITY

Every request carries its conversation identifier, parent identifier, depth and call chain. A child request is built from the handler's call context, so depth and chain are derived and cannot be forged by the handler. The four safeguards are the depth limit, the request limit per conversation, cycle detection and duplicate detection. The default limits are a maximum depth of five, at most twenty-five requests in a conversation, a request timeout of ninety seconds and a peer timeout of twenty-five seconds, all configurable. Only timeouts and transport errors are retried, with exponential backoff and the same request identifier; the orchestrator sends its full analyses without retries because they are expensive. A request therefore ends as a success, a partial result or a failure, and only retryable failures return to the running state.

4.2.4 SEQUENCE OF A DELEGATED ANALYSIS

Figure 4.2 follows a single delegation. The orchestrator sends a *sector_analysis* request to the external-sector agent. While loading its own data, that agent asks the monetary agent for the repo rate through a peer request one level deeper. The server checks the declared edge, runs the peer handler and returns the indicator signal with its provenance. The external-sector agent then completes its analysis and returns a response whose sources include the peer's original source record.

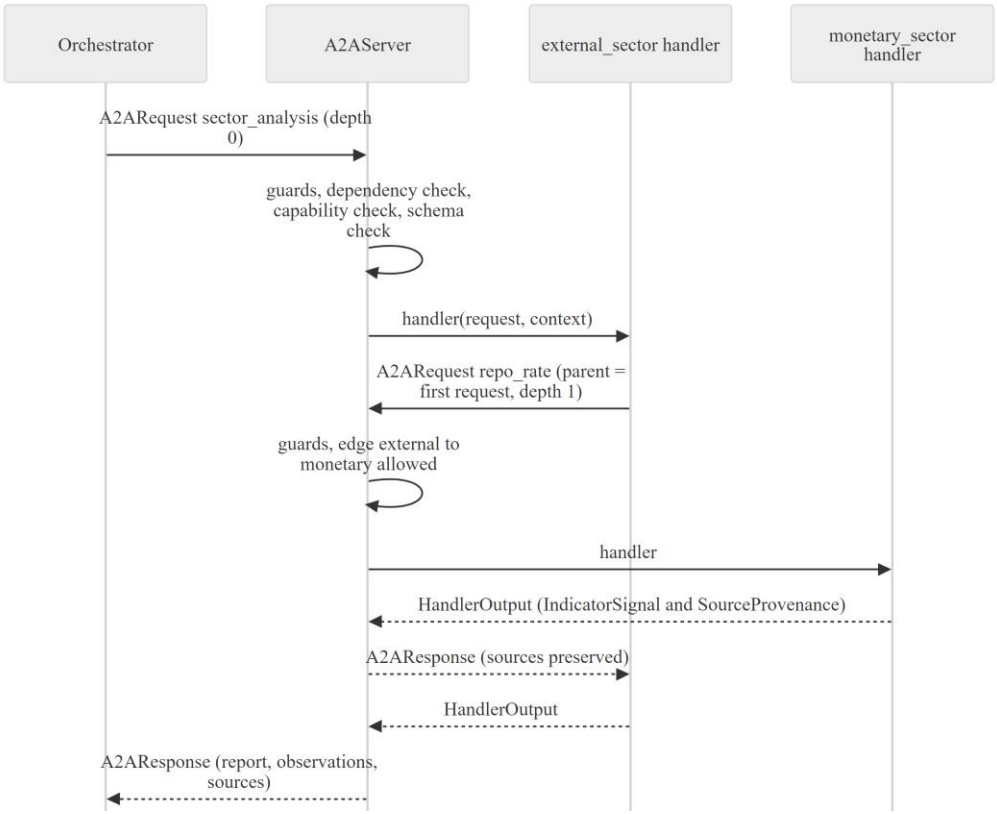


Figure 4.2: Sequence of an A2A delegation in which a sector agent requests a peer indicator.

4.3 AGENT CARDS, REGISTRY AND ROUTING

Each agent publishes an Agent Card that records its identifier, capabilities, skills, supported tasks, input and output schema names and metadata such as its sector and routing keywords. Cards are registered in one place, *registry_bootstrap.py*, which is the only module in *core* that names sector packages. The server appends a task to the card when a handler is registered, so a card always matches what is executable.

The router scores each card's capabilities, skill tags and keywords against the query. A phrase advertised by several agents counts for one over the number of agents advertising it, so a shared word such as “rbi” does not pull unrelated sectors into the answer. A language model is consulted only when no card matches distinctively, and its answer is intersected with the registered agent identifiers, so the model can select receivers but cannot bypass the protocol.

4.4 CROSS-SECTOR EXCHANGES

The orchestrator is unrestricted, but sector agents may only exchange the seven declared peer tasks listed in Table 4.3. The finance, fiscal and external agents request their peers while loading their own data through the helper

peers.gather_peer_context. A peer failure degrades to an "unavailable" line in the report and never fails the caller.

Table 4.3: Declared sector-to-sector A2A dependencies

Consumer	Provider	Task	Rationale
finance_sector	monetary_sector	repo_rate	Lending-rate spreads are measured against the policy rate.
finance_sector	prices_sector	cpi_headline	Real lending and deposit rates need headline CPI.
fiscal_sector	monetary_sector	repo_rate	Interest cost of government borrowing.
fiscal_sector	prices_sector	cpi_headline	Real deficit and revenue buoyancy.
fiscal_sector	finance_sector	bank_credit_growth	Crowding out of private credit.
external_sector	monetary_sector	repo_rate	Rate differentials drive flows and rupee pressure.
external_sector	prices_sector	cpi_headline	Imported inflation and the real exchange rate.

4.5 THE MCP DATA LAYER

Every sector exposes its data through a FastMCP server defined in *mcp_server.py*. Each server offers typed tools whose parameters are described by Pydantic models, so a client can discover and validate calls. Behind the tools, the sector's client module fetches from official sources using asynchronous HTTP with exponential-backoff retries. The sources include the MoSPI e-Sankhyiki MCP service (Ministry of Statistics and Programme Implementation, n.d.) for national accounts, industrial production, consumer prices and wholesale prices; RBI DBIE tables (Reserve Bank of India, n.d.) for rates, credit and asset quality; Yahoo Finance for market data; and budget and IMF series for fiscal data.

A tool follows the fallback chain in Figure 4.3. It first tries the live official source. If that fails, it returns the latest snapshot stored in the sector's DuckDB, tagged as cached. If neither is available it returns a structured result with the status "unavailable" and a reason. A tool never invents a number. Every result carries the value, unit, observation period, data status and a citation giving the source authority, table and retrieval time.

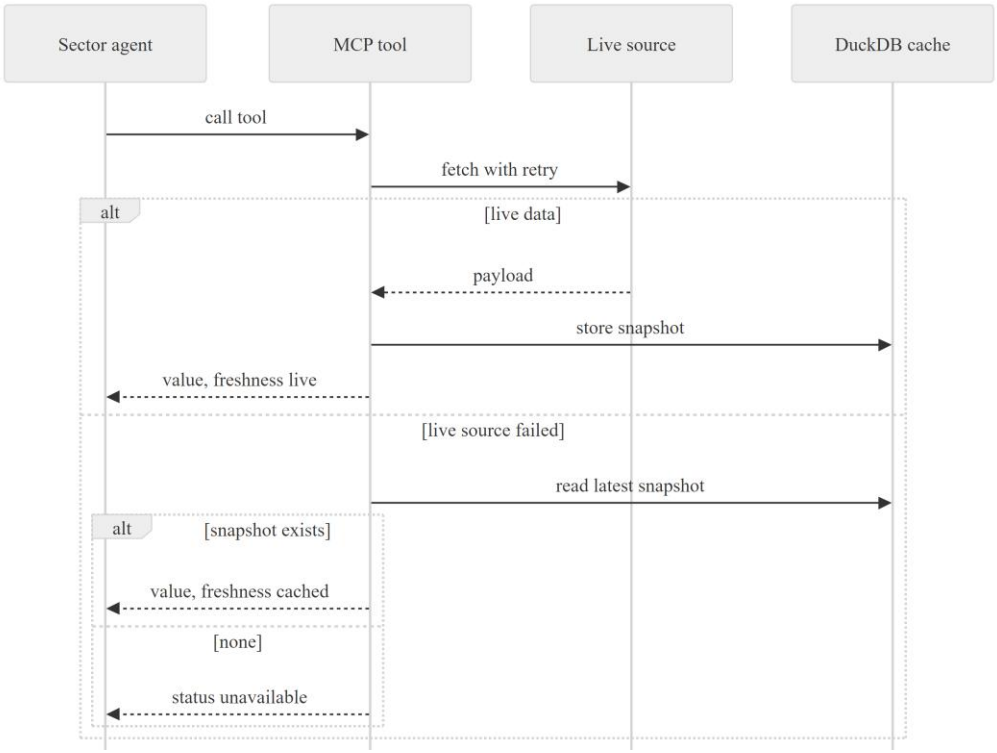


Figure 4.3: An agent retrieving data through its own FastMCP server, with the fallback order.

4.6 PROVENANCE ACROSS THE PROTOCOLS

Provenance is preserved at every step (Figure 4.4). The MCP tool result carries the source citation. The handler converts the sector's citation records into *SourceProvenance* objects; where a sector provides only a markdown report, the conversion records an artifact-level reference (agent, artifact name and content hash) instead of inventing fields. The server returns the sources unchanged. The orchestrator aggregates observations, citations and sources; each observation receives a SHA-256-based provenance hash computed from its indicator, period, value, unit and status. The final report prints the hash in its indicator matrix, and the web interface opens an evidence drawer from the same record.

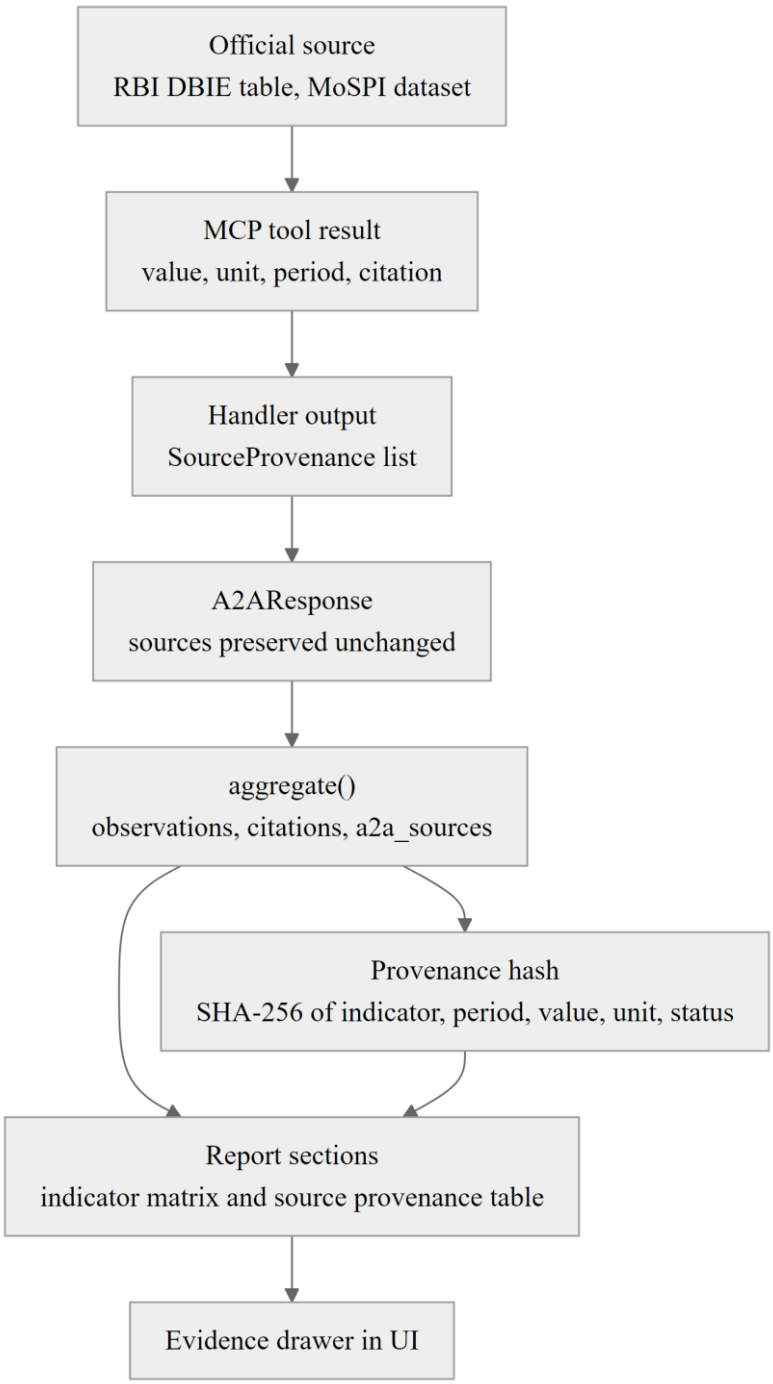


Figure 4.4: Propagation of provenance from the official source to the final report.

4.7 TRACEABILITY

A trace store records every request of a conversation. It can reconstruct the delegation tree and exposes it through *GET /a2a/v1/traces/{conversation_id}*; the chat interface streams one step event per request, and the web application renders the tree with timings. Each interaction is also logged on one line with the request identifier, parties, task, depth, status, error codes, number of sources and duration. Traces are held in memory and bounded; persisting them is listed as future work.

Chapter 5
Implementation

5.1 INTRODUCTION

This chapter describes how the design was implemented: the orchestrator, the chat routing, the sector agents and their data pipelines, the offline fallbacks and the resilience mechanisms. The implementation consists of about 37,000 lines of Python and 11,300 lines of TypeScript, excluding tests.

5.2 DEVELOPMENT ENVIRONMENT

Table 5.1 summarises the development and runtime environment.

Table 5.1: Development and runtime environment

Item	Specification
Operating system	Windows 11 (development); the code is platform independent
Backend	Python 3.12 managed with uv and a lock file; FastAPI, FastMCP, LangGraph, Pydantic v2
Data	DuckDB, SQLite, pandas, NumPy, statsmodels, NetworkX; optional Neo4j driver
Frontend	Node.js 22, React 18, TypeScript (strict), Vite 5, Tailwind CSS 3
Models	Groq-hosted open-weight models selected by FAST_MODEL and REASONING_MODEL settings
Testing	pytest with asyncio mode auto; TypeScript compilation as the frontend check
Source control	Git; development history from 16 July to 11 October 2026

5.3 THE ORCHESTRATOR

The orchestrator is a LangGraph state graph of four nodes (Figure 5.1), each reading and writing keys of a typed *OrchestratorState*.

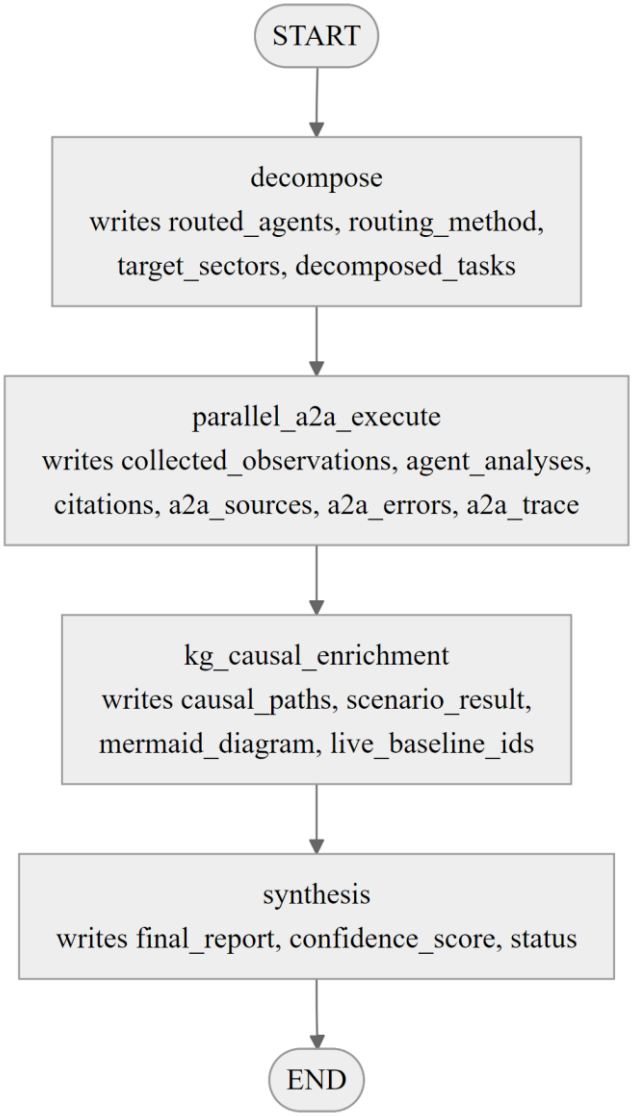


Figure 5.1: The four-node LangGraph orchestrator and the state keys written by each node.

5.3.1 DECOMPOSE

The decomposition node routes the query over the registered Agent Cards using the router described in Section 4.3. It records the selected agents, the routing method and the sectors they belong to.

5.3.2 PARALLEL A2A EXECUTION

The execution node sends one *sector_analysis* request to each selected agent concurrently. A response may include up to three follow-up requests, which are sent in one bounded second round. The aggregation step validates each response and extracts observations and citations from JSON artifacts, or keeps the report text with an artifact-level source record for agents that return markdown. Failures are appended

to the list of A2A errors and the run status becomes partial; the run continues with the data that is available.

5.3.3 KNOWLEDGE-GRAPH ENRICHMENT

The enrichment node keeps the indicators of the collected observations that are nodes of the knowledge graph and queries the shortest transmission path for every ordered pair. If the request included a scenario shock it runs the scenario engine. First-usable live observations from the sector agents replace the default baselines, so the report cannot contradict the retrieved data, and the report states which indicators used live baselines. A Mermaid diagram of the first five graph nodes is produced for the report.

5.3.4 SYNTHESIS

The synthesis node builds the final report. The language model writes only the executive synthesis, using a prompt that contains the observations, causal paths with relation identifiers, scenario outputs and the list of agents that failed. Sections two to six are assembled deterministically from structured state (Figure 5.2): the indicator matrix with provenance hashes, the causal diagram and path table, the scenario table, the source provenance table and the methodological notes. If no language model is available, the executive synthesis is a deterministic summary of the evidence, so the evidence sections never depend on model output.

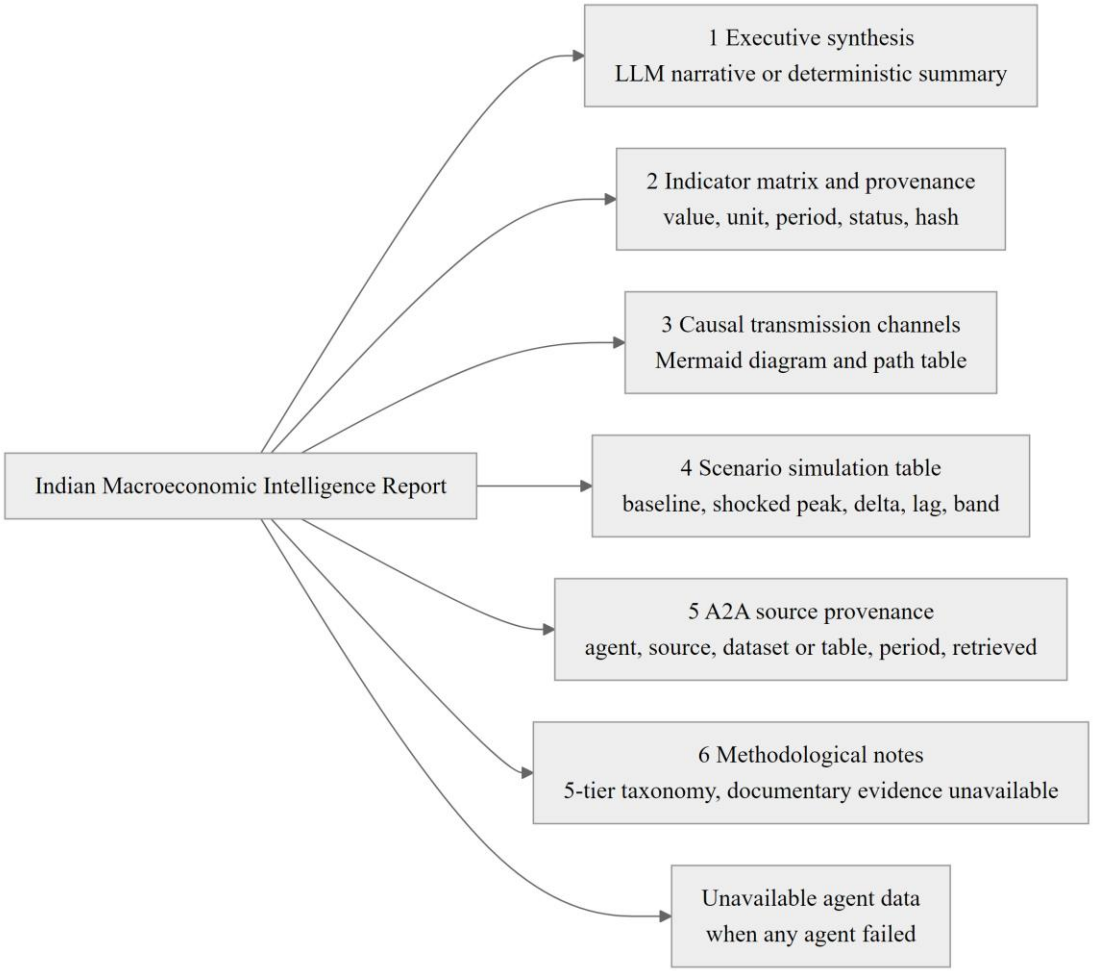


Figure 5.2: Sections of the cited intelligence report produced by the synthesis node.

5.4 CHAT INTERFACE AND ROUTING

The chat router streams progress and text with Server-Sent Events. The selected agent determines the route. A direct sector chat calls that sector's agent node and bypasses the orchestrator. When the orchestrator is selected but the query is clearly a single prices question, it is redirected to the prices agent. All other queries run the orchestrator graph. The final event carries the report, citations, the A2A summary, a consensus summary and a model card.

5.5 SECTOR AGENTS AND DATA ACQUISITION

All ten sector agents share the pipeline in Figure 5.3. Each fetches from official sources through its own client, normalises and validates the payload, stores it with parameterised SQL and exposes it through an MCP tool together with a citation and a freshness flag. Figure 5.4 lists the primary sources of each sector.

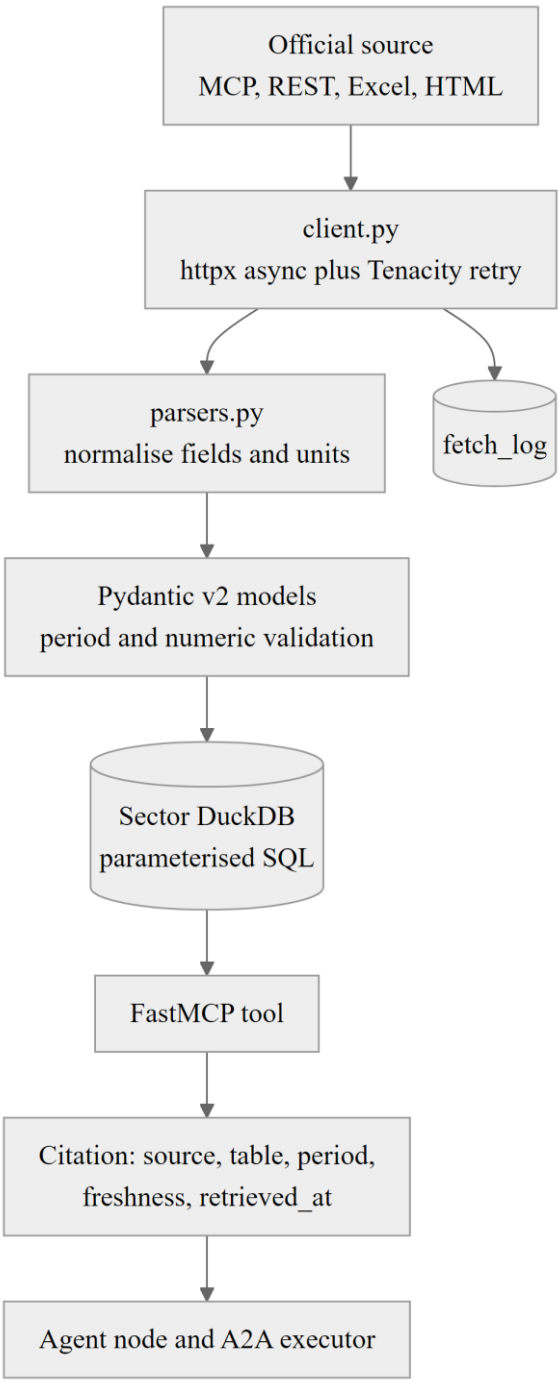


Figure 5.3: Data acquisition pipeline shared by the sector agents.

The prices, finance and several other agents combine the official observations with market proxies and news as context. A direct agent follows a fixed reasoning pipeline. Official observations are fetched first; market proxies and news are fetched concurrently as context and are never treated as authoritative data; the language model receives the consolidated dataset and must cite periods and sources. Queries outside the sector's domain are politely declined with a pointer to the correct agent.

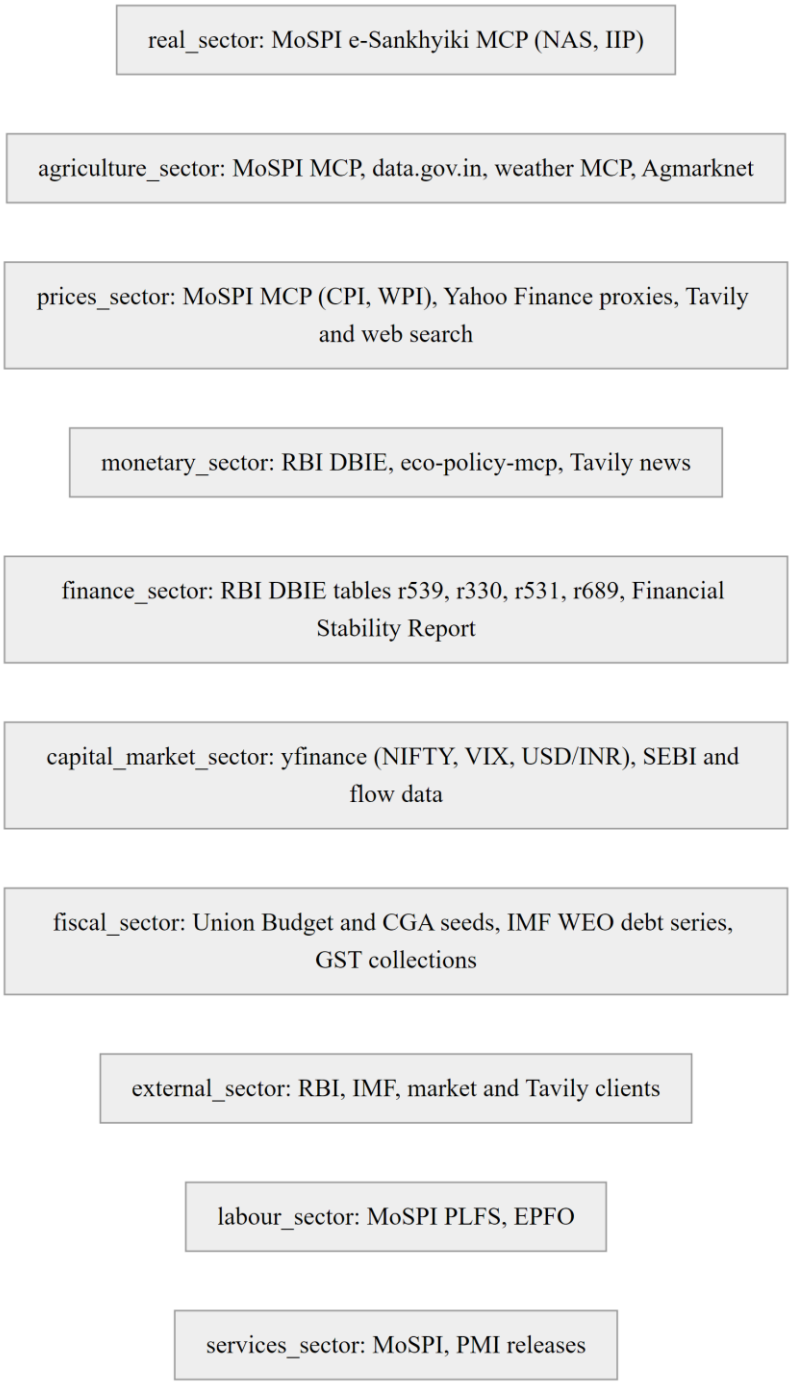


Figure 5.4: Primary data sources feeding each sector agent.

5.6 STORAGE

Sector data is stored in DuckDB tables whose names follow the indicators they hold. Each sector's schema creates tables named after the indicators it holds, for example policy rates, money supply, system liquidity and monetary stance for the monetary sector, or union fiscal deficit, general government debt and GST collections for the fiscal sector. The canonical store keeps the indicator registry, canonical observations, the daily briefs and a news cache. SQL statements use parameter placeholders for all

values. Curated baseline snapshots (for example Union Budget aggregates) are seeded into the sector stores with their publication references and are tagged as cached.

The shared ingestion pipeline of the core data layer registers one adapter per canonical indicator, validates numeric ranges and period formats, normalises to a canonical observation, attaches the provenance hash and stores the result.

5.7 ROBUSTNESS AND OFFLINE OPERATION

The platform is designed to run in degraded conditions. External calls use exponential-backoff retries. Every language-model stage has a deterministic alternative built from the same structured evidence: routing falls back to card scoring, synthesis to a counted summary, the daily brief to a template and the simulated committee to rule-based votes. When a data source fails, the tool serves a cached snapshot or an unavailable status, and the failure is recorded as an A2A error and listed in the report. The test suite exercises the no-key path explicitly, because mocking the language model once hid an import error in the real client.

5.8 CODE ORGANISATION AND STATISTICS

Table 5.2 summarises the size of the implementation. Test code is excluded from the line counts.

Table 5.2: Source lines by component (tests excluded, 12 October 2026)

Component	Lines
Frontend (TypeScript)	11,287
core (orchestrator, protocols, graph, research, briefing, verification)	6,778
api (routers and helpers)	2,877
Ten sector packages (between 1,872 and 3,274 lines each)	27,257
Backend total of the packages above	36,912

Chapter 6
Knowledge Graph and Scenario Engine

6.1 INTRODUCTION

Sector agents report individual indicators. A question such as “what does an oil shock do to policy rates?” is about how those indicators relate to each other, and that is what this chapter covers. It describes the causal knowledge graph that stores the relationships, the queries the orchestrator runs on it, and the scenario engine that pushes a shock through it. All numbers here come from the implemented engine, run on 12 October 2026.

6.2 ONTOLOGY OF INDICATORS AND TRANSMISSION EDGES

The ontology in *core/knowledge_graph/ontology.py* holds twenty canonical indicators and twelve directed relationships. Each indicator has an identifier of the form *in.macro.<domain>.<name>* and sits in one of eight domains: real economy, prices and inflation, monetary and banking, fiscal, external, capital markets, agriculture and labour. Figure 6.1 shows the graph. Five indicators (NIFTY 50, India VIX, forex reserves, debt-to-GDP and EPFO additions) have no edges yet. They are valid nodes, and relationships can be added to them as the ontology grows.

Relationship identifiers look like *rel-<8 hex digits>*. Each comes from a version-5 UUID of the source, target and type, so it survives restarts and works as a provenance key. Table 6.1 lists all edges. Relationship types follow a five-tier taxonomy, from qualitative theory to a structural causal model.

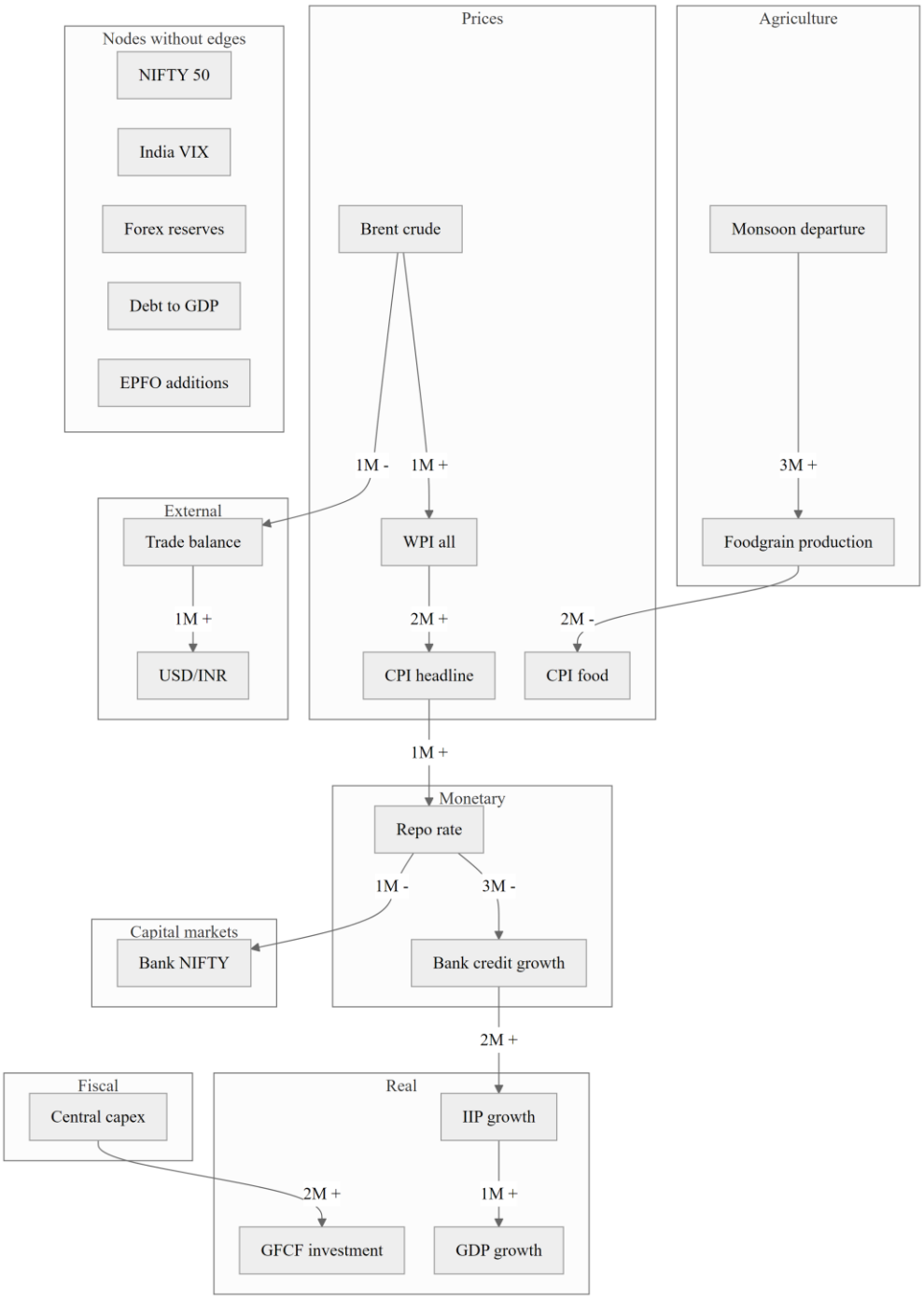


Figure 6.1: Causal transmission graph of 20 indicators connected by 12 typed edges (lag in months and sign).

Table 6.1: The twelve causal edges of the knowledge graph

Source	Target	Type	Lag (months)	Sign	Confidence
Brent crude	WPI	Lagged relationship	1	+	0.95
WPI	CPI headline	Granger predictive	2	+	0.90
CPI headline	Repo rate	Structural causal model	1	+	0.95
Repo rate	Bank credit growth	Granger predictive	3	−	0.88
Bank credit growth	IIP growth	Statistical association	2	+	0.82
IIP growth	GDP growth	Structural causal model	1	+	0.98
Brent crude	Trade balance	Theory	1	−	0.92
Trade balance	USD/INR	Statistical association	1	+	0.86
Repo rate	Bank NIFTY	Lagged relationship	1	−	0.85
Central capex	GFCF investment	Theory	2	+	0.90
Monsoon departure	Foodgrain production	Theory	3	+	0.92
Foodgrain production	CPI food	Statistical association	2	−	0.89

The ontology validates itself when it loads. It rejects self-loops, unknown endpoints, duplicate identifiers, malformed identifiers and any sign other than plus or minus. The confidences and p-values on the edges are curated priors that the designer typed in. Section 7.5 explains how they are audited against data, and Chapter 9 reports what the audit found.

6.3 GRAPH ENGINE AND QUERIES

NetworkXGraphEngine builds an in-memory directed multigraph from the ontology and supports three queries. The shortest transmission path between two indicators is the path with the fewest hops; if two parallel edges exist, the one with higher confidence is used. The downstream impact query finds every descendant of a shocked indicator and reports its hop count, total lag (the sum of edge lags), cumulative confidence (the product of edge confidences) and the chain of mechanism descriptions. The causal neighbourhood query returns indicators within a given depth. The engine can also produce Mermaid diagrams for the reports.

Figure 6.2 summarises the structure, storage and query services of the graph. A Neo4j mirror can store the same nodes and edges for persistence. It is off unless a URI is

configured. When it is on, the ontology is synchronised once at gateway start by an idempotent upsert, with three retry attempts. NetworkX remains the query engine of the orchestrator.

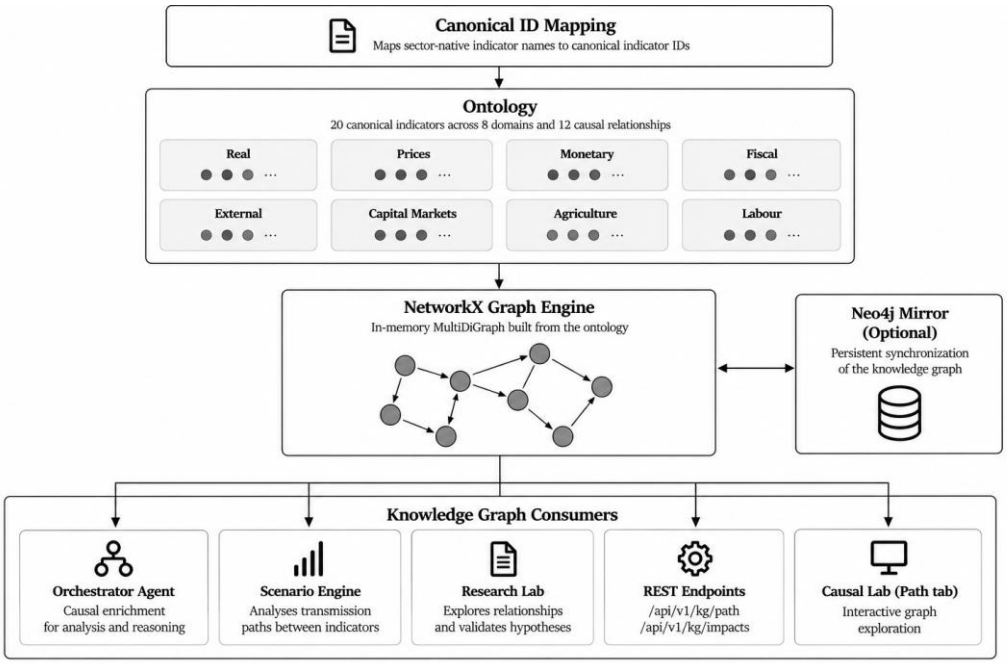


Figure 6.2: Structure, storage and query services of the knowledge graph.

6.4 SCENARIO ENGINE

The scenario engine sends a shock along the shortest path from the shocked indicator to each downstream indicator. For a path with hops h, edge signs s_h and confidences c_h, the path multiplier and the target delta are:

$$M(path) = \Pi \text{ over hops } h \text{ of } (s_h \times 0.55 \times c_h) \qquad (1)$$

$$\Delta(target) = shock \times M(path) \times 0.15 \qquad (2)$$

Figure 6.3 gives an overview of how a shock travels along the causal graph and how the result is used downstream. The shocked peak is the baseline plus the delta, and the engine reports a band of plus or minus 25 per cent of the delta. The constants 0.55 (per-hop attenuation) and 0.15 (global scale) are design parameters, not estimates from data. The lag of an effect is the sum of the edge lags, and the cumulative confidence is the product of the edge confidences. The engine is therefore a transparent heuristic that shows how a shock moves through the graph. It is not a vector autoregression or an impulse-response estimate, the deltas are in each indicator's own unit, and the band is not a statistical interval. That is why every scenario output is labelled as simulated.

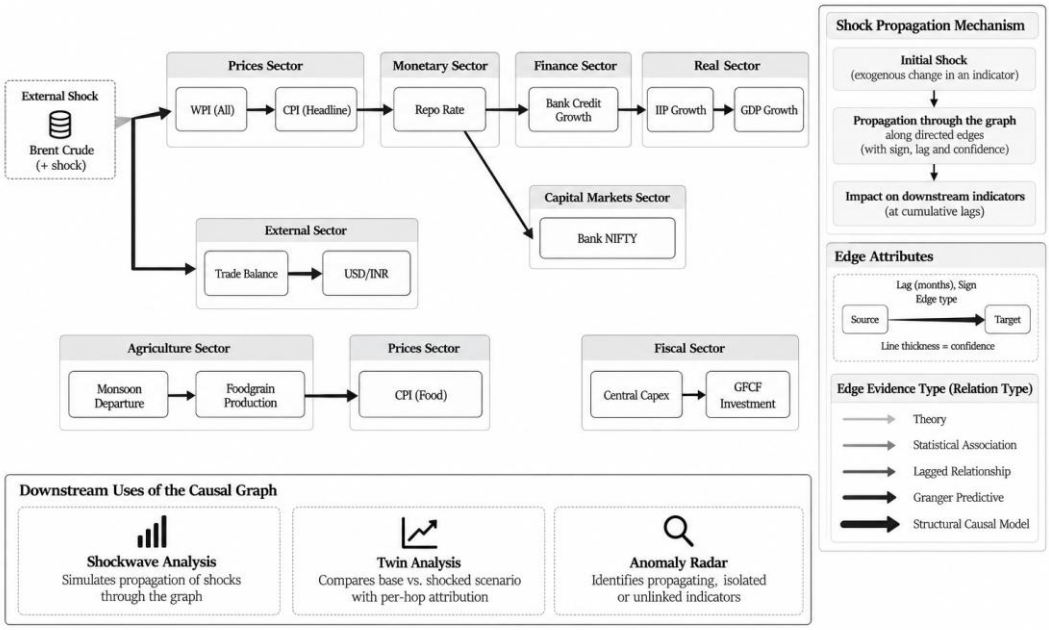


Figure 6.3: Causal transmission graph and downstream uses of shock propagation.

6.5 WORKED EXAMPLE: CRUDE OIL SHOCK

Table 6.2 shows what the engine returns for a shock of plus 20 US dollars per barrel on Brent crude, with the default baselines. The first hop shows the arithmetic: 20 × (0.55 × 0.95) × 0.15 = 1.57. Added to the wholesale price inflation baseline of 2.45, this gives 4.02.

Table 6.2: Simulated transmission of a USD 20 per barrel Brent increase (default baselines)

Indicator	Baseline	Shocked peak	Delta	Lag (months)	Cumulative confidence
WPI all commodities	2.45	4.02	+1.57	1	0.95
Merchandise trade balance	−22.50	−24.02	−1.52	1	0.92
CPI headline	4.26	5.04	+0.78	3	0.85
USD/INR	86.40	85.68	−0.72	2	0.79
Repo rate	6.25	6.66	+0.41	4	0.81
Bank NIFTY	51500	51499.81	−0.19	5	0.69
Bank credit growth	13.80	13.60	−0.20	7	0.71
IIP growth	4.20	4.11	−0.09	9	0.59
GDP growth	5.40	5.35	−0.05	10	0.57

Figure 6.4 shows the monthly trajectories that the research lab produces for the same shock. Each series stays at its baseline until the cumulative lag of its path is reached, then steps to its delta. Wholesale prices respond first. The output effects come nine to ten months later, and they are much smaller, because every hop multiplies the signal by less than one.

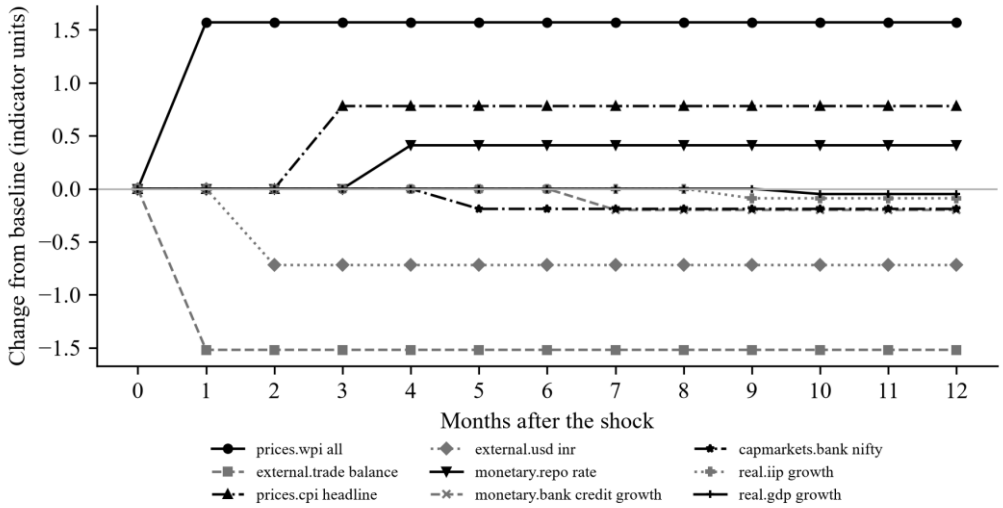


Figure 6.4: Simulated monthly change from baseline after the shock (step response, produced by the Shockwave module).

Two things in this example need care when interpreting it. First, cumulative confidence falls with every hop, from 0.95 after one hop to 0.57 after six, so effects far along a path should be trusted less. Second, USD/INR falls after the oil price rise in this run, which is economically implausible. The cause is the plus sign on the edge from the trade balance to USD/INR. A wider trade deficit lowers the trade balance, and with a plus sign that pulls the rupee-per-dollar rate down. The code already marks this edge as under review. The example stays in because it shows the evidence labelling doing its job of exposing problems like this. Fixing the sign is the first item of future work.

Inside the orchestrator, the same shock runs on live data, and the baselines of the indicators the sector agents retrieved replace the defaults. In the end-to-end run of Section 9.4, the live CPI of 5.54 per cent (September 2026) and repo rate of 5.25 per cent took the place of the defaults. For the same deltas of +0.78 and +0.41, the simulated peaks were 6.32 for CPI and 5.66 for the repo rate.

Chapter 7

Research Lab, Daily Brief and Claim Verification

7.1 INTRODUCTION

Beyond answering questions, the platform has tools for inspecting and testing its own model. The research lab (*core/research*) is made of pure functions over the frozen knowledge graph, the scenario engine and the series held by the dashboard helpers, and it never changes engine state. The daily brief (*core/briefing*) summarises the state of the economy. The claim verification library (*core/verification*) checks generated statements against evidence. Figure 7.1 shows how the lab modules connect to endpoints and interface tabs. Every payload carries a label (SIM, CACHED or heuristic), so the interface can tell simulated, cached and measured values apart.

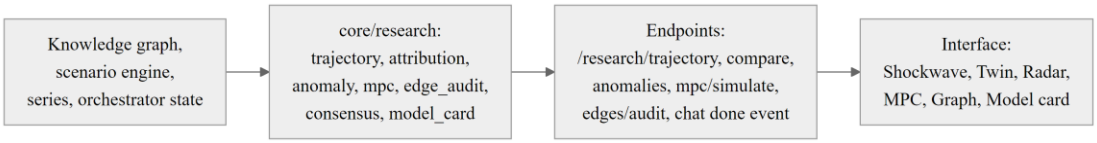


Figure 7.1: Research lab modules, API endpoints and user interface tabs.

7.2 SHOCKWAVE: MONTHLY TRAJECTORIES

The scenario engine returns one peak delta and one lag per node. The trajectory module turns these into monthly series over a horizon of one to sixty months. A series stays at the baseline until the lag month. After that it either steps to the peak or decays exponentially with a chosen half-life, following baseline + delta × 0.5^((t − lag) / half-life). The module also returns the edges on each path, with their evidence class and review notes. The notes say that deltas are in each indicator's unit and that the band is not a statistical interval.

7.3 TWIN: BASE WORLD VERSUS SHOCKED WORLD

The twin module runs the engine twice, once with zero shock and once with the requested shock, and compares the two results node by node. For each target it also splits the effect across the hops of its path. A hop's share is proportional to the logarithm of its attenuation factor (0.55 times its confidence). The hop with the largest share is therefore the weakest link, the one that attenuates the signal most. Edges with a negative sign on the path are counted as sign flips. Figure 7.2 shows the split for the path from Brent crude to GDP growth. The shares are close to each other (15 to 19 per

cent), and the weakest link is the edge from bank credit growth to IIP growth, whose confidence of 0.82 is the lowest on the path.

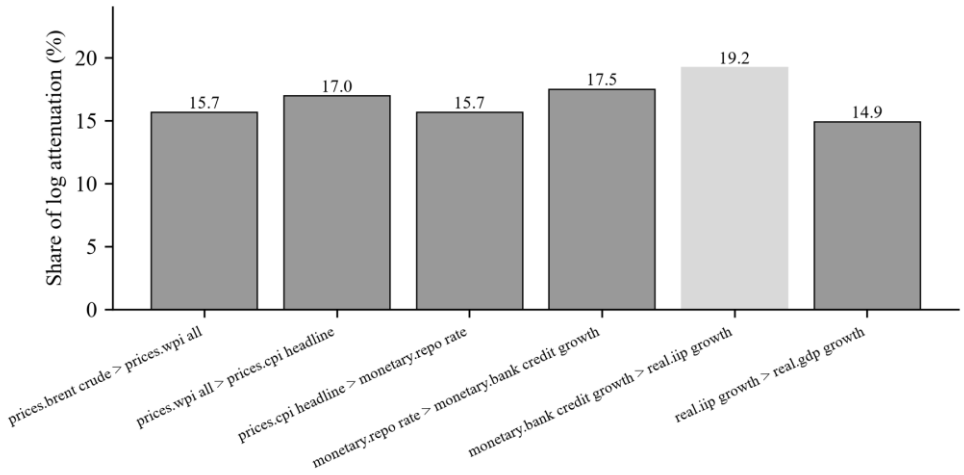


Figure 7.2: Per-hop share of attenuation along the path from Brent crude to GDP growth (Twin module, real output).

7.4 ANOMALY RADAR

The anomaly radar flags unusual latest readings and uses the graph to decide whether an anomaly stands alone or is part of a propagating pattern. It works on every series with at least eight points. The latest value is compared with a trailing window of up to twelve observations, leaving the latest point out of its own baseline, and a z-score is computed. An absolute z-score above 2.0 counts as an anomaly. The module then looks at the graph neighbours of the indicator, upstream and downstream. If at least one neighbour with data is also anomalous, the anomaly is classed as propagating and the culprit path of relation identifiers is returned. If neighbours have data but none is anomalous, it is isolated. If no neighbour has data, it is unlinked. Isolated and unlinked anomalies are flagged for source verification, since they may be data errors. A simulation mode can inject a synthetic shock into a series to demonstrate propagation.

7.5 EDGE EVIDENCE AUDIT

The edge evidence audit checks whether the seeded edges of the graph are backed by data (Figure 7.3). For each edge, it aligns the source and target series by calendar month. If fewer than ten months overlap, the edge stays untestable. Otherwise a bivariate Granger causality test with a maximum lag of two is run, and the edge is labelled EMPIRICAL, with its p-value and optimal lag. Edges that cannot be tested

stay SEEDED. Results are cached for five minutes. The audit keeps evidence typed in by a designer separate from evidence estimated from data. The interface shows the label on every edge.

The method has limits, and the report says so plainly. Ten months is a low minimum for reliable Granger inference. The series are not tested for stationarity, and there is no correction for testing many edges at once. Chapter 10 describes the improvements that are planned. Section 9.5 reports what the audit found when it was run on the data available now.

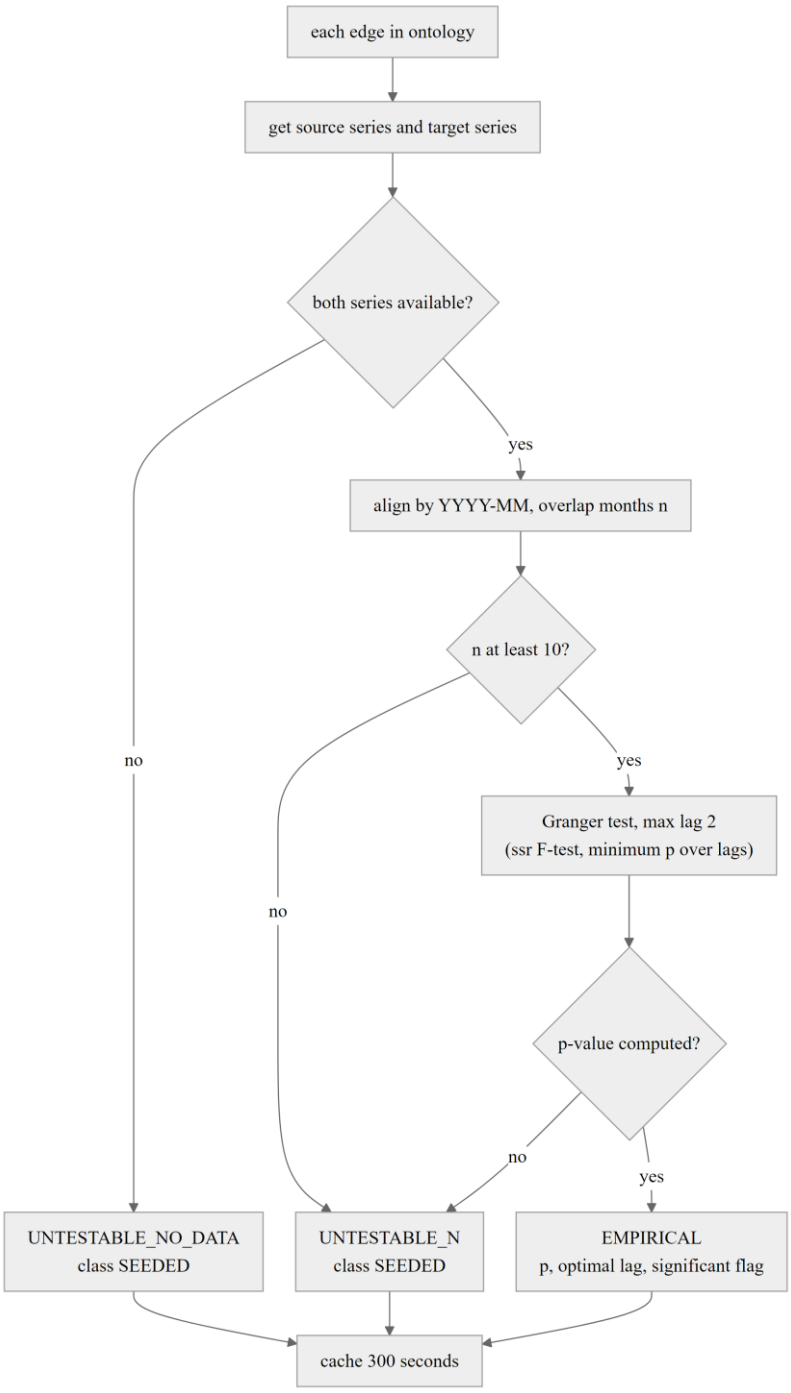


Figure 7.3: Edge evidence audit: seeded priors tested against live series.

7.6 SIMULATED MONETARY POLICY COMMITTEE

The simulated committee has six seats with archetype personas: a centrist governor, a hawkish deputy governor, a centrist officer, one hawkish external member and two dovish external members. An evidence pack gathers the latest CPI, repo rate, GDP growth, USD/INR and bank credit growth from the canonical store and the dashboard series. Each persona first votes by a deterministic rule. A hawk hikes when CPI is above 6 per cent and otherwise holds. A dove cuts when CPI is below 2 per cent, or

when growth is weakening and CPI is not above 6 per cent, and otherwise holds. A centrist hikes above 6 per cent, cuts below 2 per cent, cuts when growth is weakening and CPI is below 4 per cent, and otherwise holds. The thresholds follow the inflation-targeting band: a 4 per cent target with a tolerance of 2 per cent on either side.

Figure 7.4 summarises the procedure. If a language model key is available, each of the three persona types also casts a vote as structured JSON. The vote must quote a number and cite an indicator from the evidence pack. An answer that fails this check is discarded and the rule vote stands. The six seats share the three persona results, so the committee has three distinct voices, not six independent members. The decision is the majority, and the governor's vote settles a tie. The rule-only decision is always reported next to the final one, so the contribution of the language model can be measured. The meeting is always labelled as simulated and is not a forecast of an actual decision.

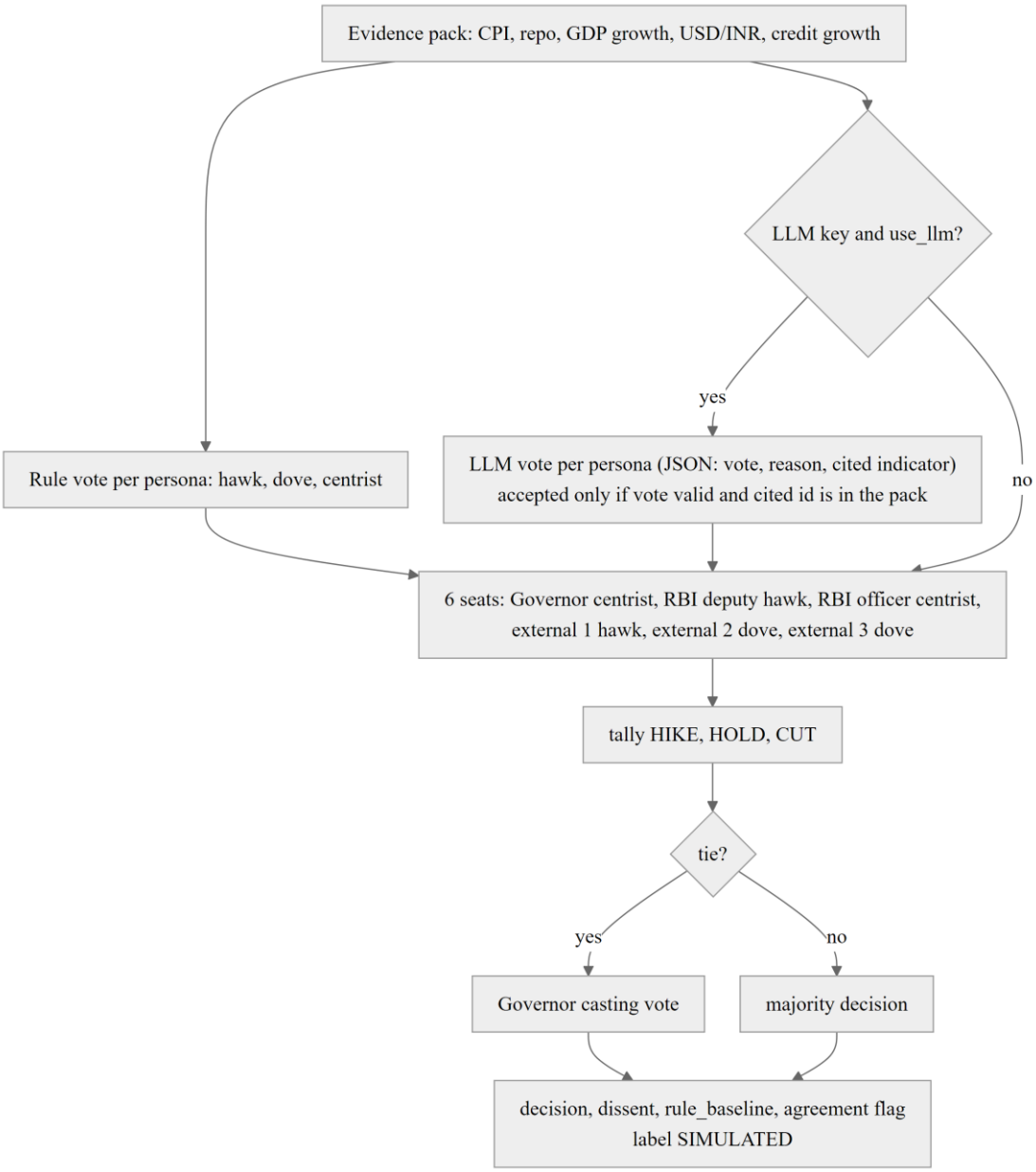


Figure 7.4: Simulation of a six-seat Monetary Policy Committee with a deterministic rule baseline.

7.7 CONSENSUS AND MODEL CARD

Every orchestrated answer carries two summaries. The consensus summary reports, for each agent, the status, duration, number of observations and number of usable observations. From these it computes three fractions: agent success (success counts as 1 and partial as 0.5), evidence cover (usable observations over all observations) and knowledge-graph cover (connected pairs over all pairs of graph indicators). Their product, with the graph cover weighted as 0.5 + 0.5 times the cover, is the consensus quality. The model card reports the evidence counts, the relations used and their evidence class, the scenario formula and its constants, an inflation regime note, and a

trust score of 0.4 times evidence cover plus 0.3 times mean edge confidence plus 0.3 times agent success. These scores are labelled heuristic. Their weights are design choices and have not been calibrated against verification outcomes.

7.8 DAILY EARLY-WARNING BRIEF

The daily brief reads telemetry for four pillars (monetary, fiscal, external and markets) and runs the anomaly scan. It fetches news with a cap of four queries and a six-hour cache, and makes at most one language model call. If the model is unavailable or returns invalid JSON, a deterministic template is used instead. Briefs are stored by Asia/Kolkata date in the canonical DuckDB store. They can be generated on demand or by an optional scheduler at 08:15, and a fifteen-minute debounce interval blocks repeated runs unless one is forced.

7.9 CLAIM VERIFICATION

The claim verification library covers the remaining gap: any figure in generated text must equal the retrieved figure. It was first built for the prices sector and is now a shared library (Figure 7.5). Retrieved observations and source passages are frozen into content-addressed evidence. The identifier of an observation is the SHA-256 digest of its canonical form, without volatile fields. Repeated retrievals of the same published value therefore share an identifier, while a revision or a change of definition gets a new one.

Claims are proposed in a strict schema with six types: an observation value, a rate change between two observations, the direction of a year-on-year price change, a reported explanation quoted from a retrieved passage, a reviewed general mechanism, and a statistical finding. Extra fields are forbidden, so a model cannot introduce a new claim type, a formula or free text. Numerical claims are recomputed with exact decimal arithmetic and compared at the displayed precision. The checks cover identifiers, finite values, monthly periods, units, geography, sector, series, source, index base, period order, conflicting observations and the requested scope. A quoted explanation is accepted only if it is a complete sentence found verbatim in the retrieved passage. Even then it is rendered as “source attributed”, and the text states that causation is not established.

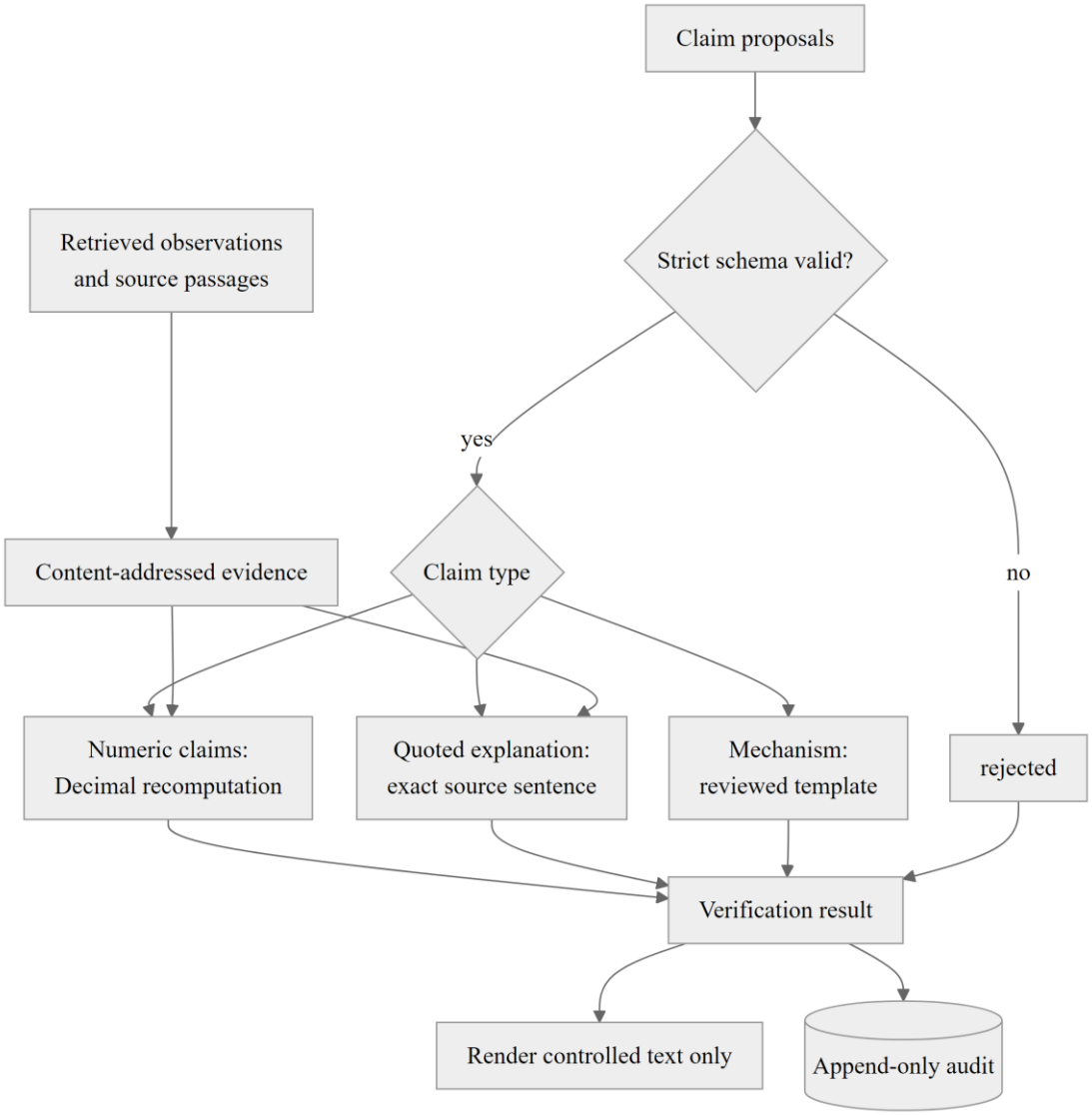


Figure 7.5: Claim verification pipeline from retrieved evidence to rendered text and audit record.

Each claim ends with one of five statuses: verified, source attributed, general mechanism, rejected, or insufficient evidence. Only the first three statuses are rendered. Rejected claims and claims with insufficient evidence are withheld and counted, and they are kept in an append-only SQLite audit together with the complete evidence snapshot, the proposals and the results. The library is covered by tests that include altered quotations, wrong arithmetic, mismatched index bases and conflicting observations.

There is one important limitation. The library is not yet connected to the answer path. The pipeline that proposed claims for the prices agent was removed from that agent, and neither the orchestrator nor any sector calls the verifier. In the planned integration (Figure 7.6), the language model returns structured claims, the verifier checks them

against the A2A evidence, only verified statements are rendered, and causal claims are hedged according to the evidence class of the edge they rely on.

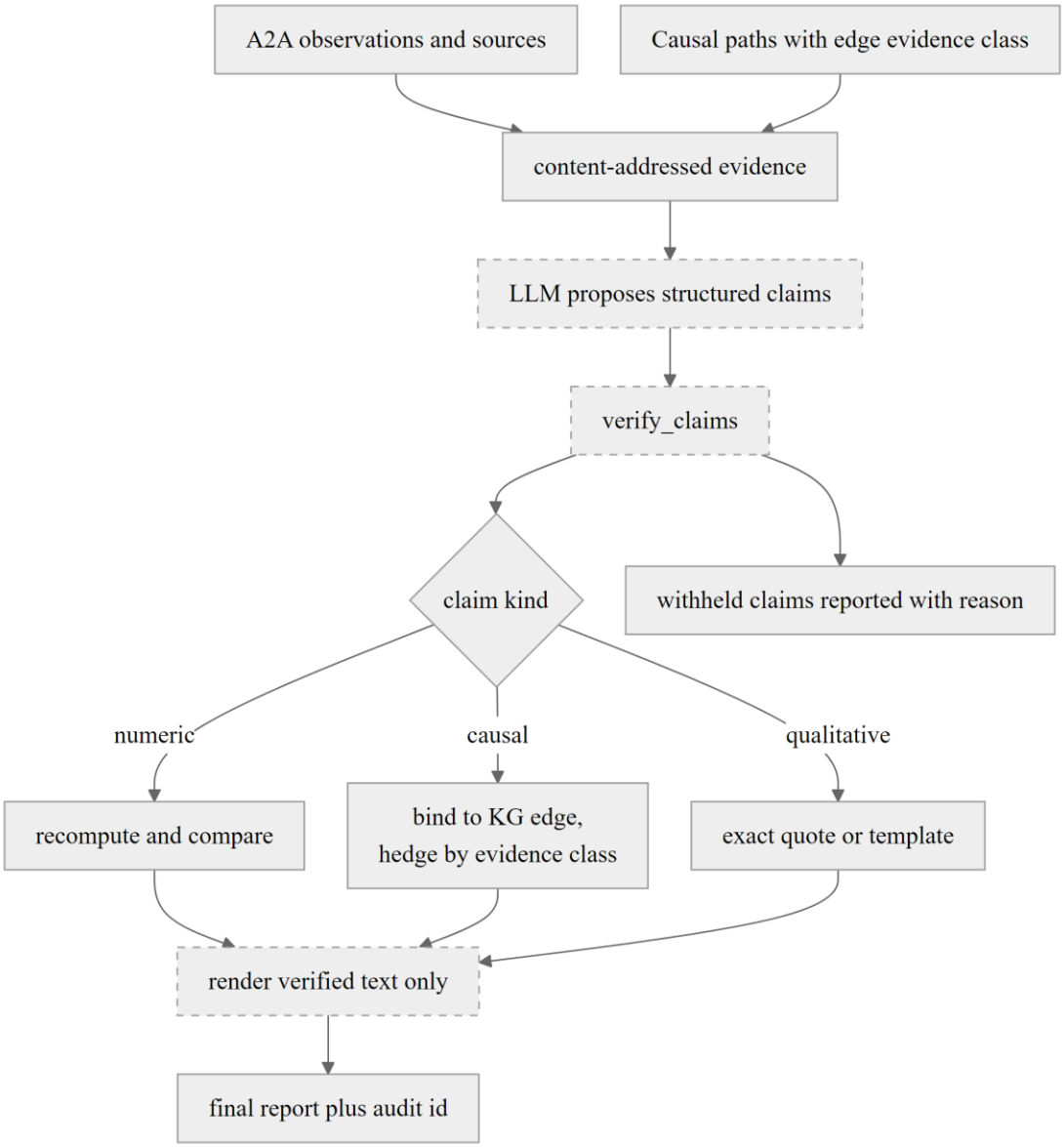


Figure 7.6: Proposed integration of the claim verifier into orchestrator synthesis (not yet implemented).

Chapter 8
User Interface and API

8.1 INTRODUCTION

This chapter describes the web application and the HTTP interface that users and other programs use to reach the platform. The screenshots were taken from the running application on 13 October 2026, with the language model and search keys disabled. The system was therefore in its deterministic offline mode, and the data shown is the sector data the platform held at that time: live where the sources responded and cached otherwise, as tagged in the interface.

8.2 FRONTEND ARCHITECTURE

The frontend is a React 18 application in strict TypeScript, bundled with Vite and styled with Tailwind CSS. The root component holds the signed-in user and the current view. A visitor who is not signed in sees the landing page. After Google sign-in, or if a stored session exists, the sidebar lets the user switch between the dashboard, the agent research workspace, the causal transmission graph with the causal lab, and the A2A agent registry. Signing out clears the stored profile.

Figure 8.1 shows the component hierarchy. The dashboard has eight sector tabs, studio widgets (scenario simulator, cross-sector studio, anomaly feed and early-warning brief) and reusable charts. The chat workspace renders streaming markdown, scrollable tables, Mermaid diagrams, the A2A trace panel, the debate graph and the model card strip. The causal lab is a tab shell around the knowledge-graph view and the four research tabs. Network requests use an *AbortController*, so a new question cancels the previous one.

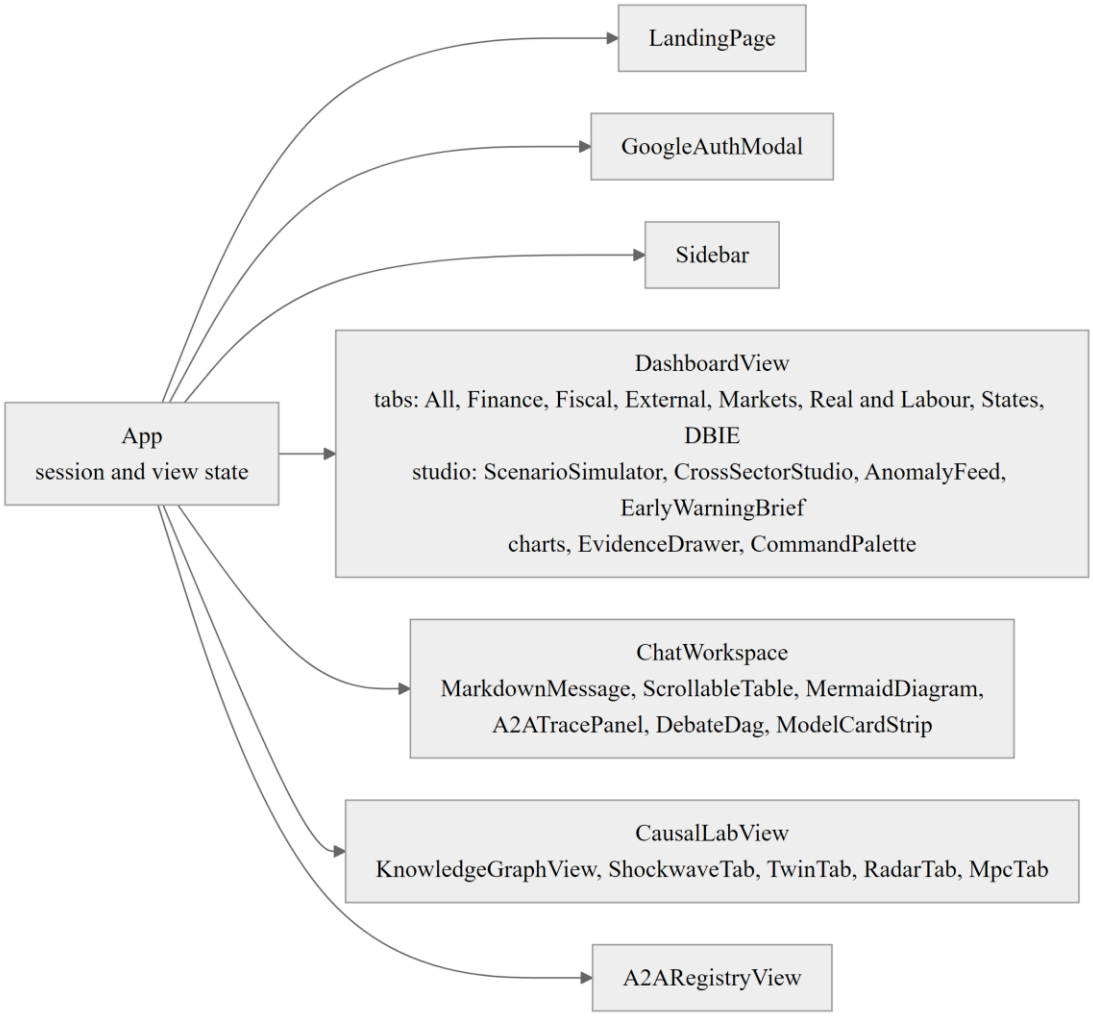


Figure 8.1: React component hierarchy of the application.

8.3 LANDING PAGE AND AUTHENTICATION

Figure 8.2 shows the landing page. It introduces the platform and its dual-protocol design and offers Google sign-in, which uses Google Identity Services. The decoded profile is kept in the browser's local storage, and there is no server-side session. This means the gateway endpoints themselves are not protected, and Chapter 9 lists that as a limitation.

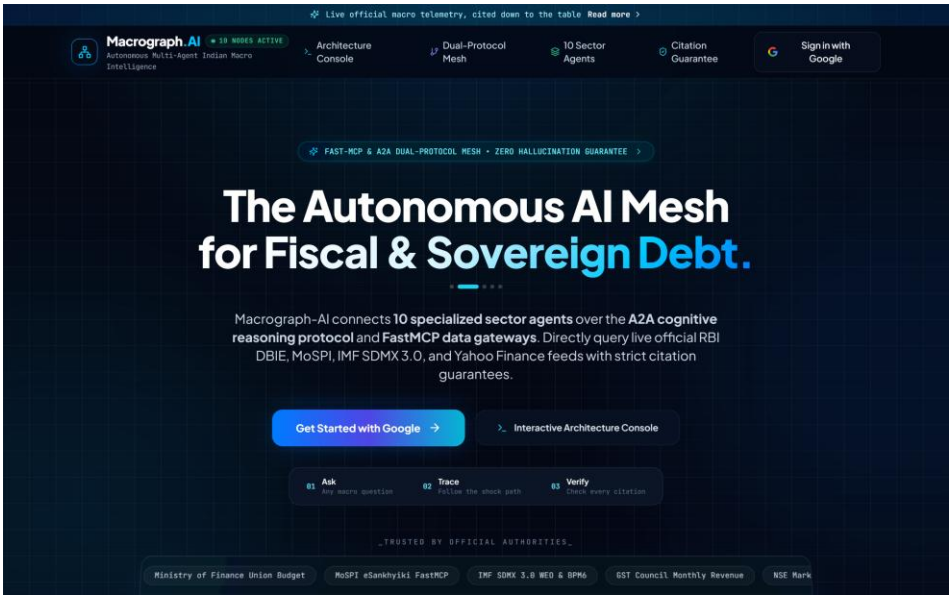


Figure 8.2: Landing page of Macrograph AI.

8.4 DASHBOARD

The dashboard pulls telemetry from the sector stores, open datasets and the RBI DBIE explorer into eight tabs. Every indicator keeps its own citation, and clicking a value opens an evidence drawer with the source, table, period and calculation. Figure 8.3 shows the first tab. It combines a ticker bar of the policy rate, yields and exchange rate with the early-warning brief panel and the cross-sector summary. The finance and banking tab (Figure 8.4) adds credit expansion, asset quality gauges and lending-rate spreads.

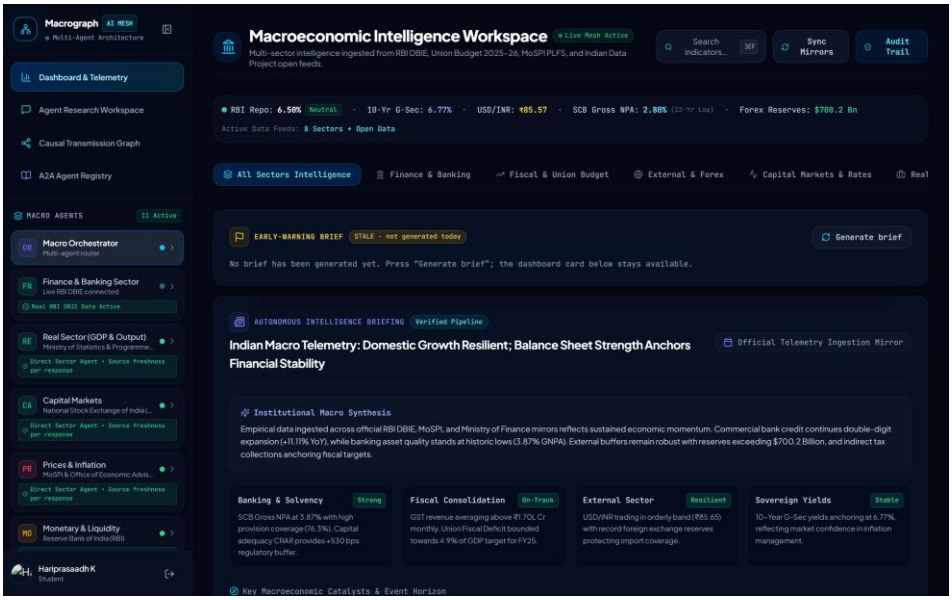


Figure 8.3: Dashboard, all-sectors tab with ticker bar and early-warning brief panel.

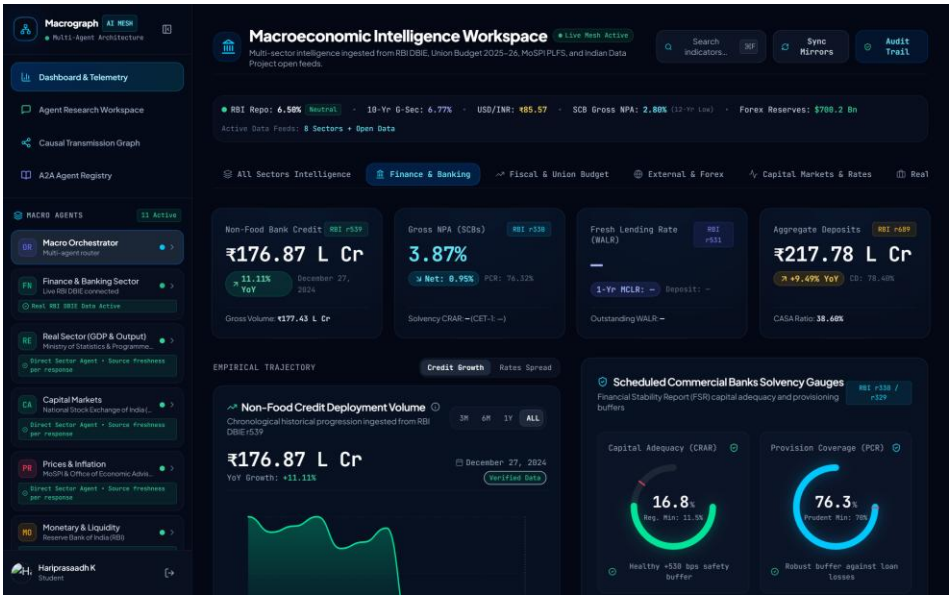


Figure 8.4: Dashboard, finance and banking tab: credit expansion, asset quality gauges and lending rates.

8.5 AGENT RESEARCH WORKSPACE

The workspace is a conversational interface to the orchestrator and to each sector agent. The sequence in Figure 8.5 shows the streaming protocol. The browser posts the message and the selected agent, then reads the response stream. Step events say which agent or tool is running and are updated in place by request identifier. Token events carry the report text. The final event carries the complete report, citations, A2A summary, consensus summary and model card, and the workspace renders the tables, causal diagram, trace panel and model card from it.

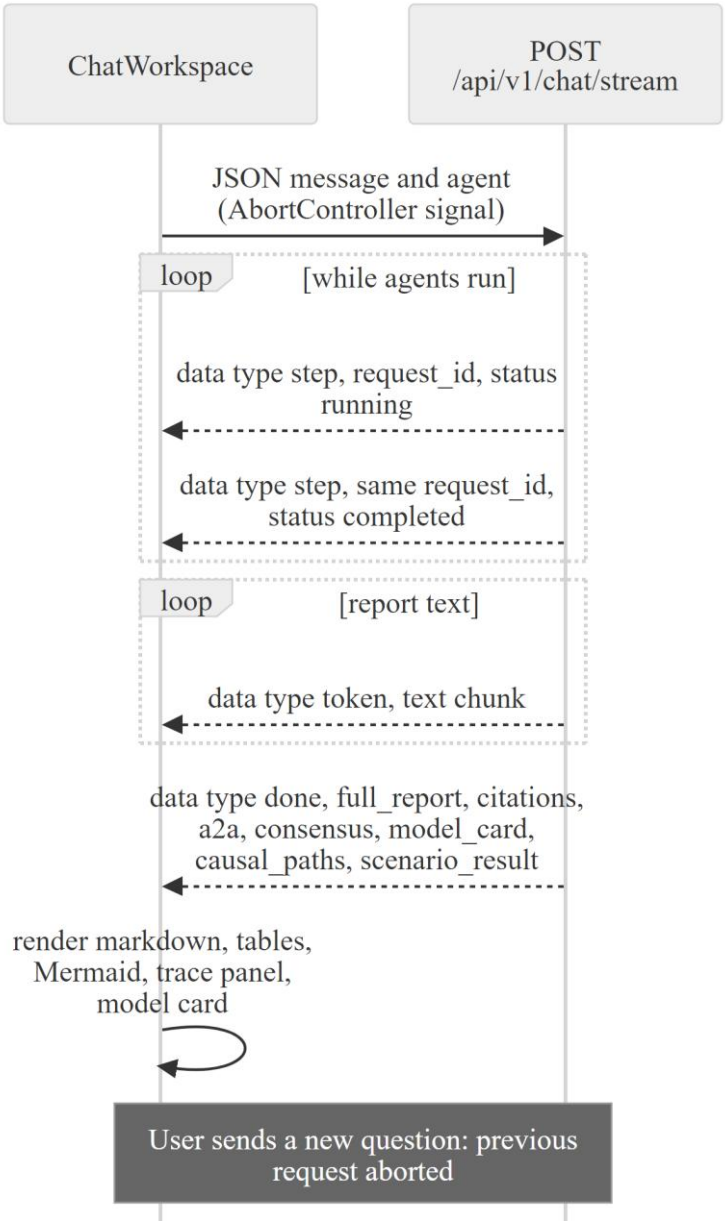


Figure 8.5: Server-Sent Events protocol between the chat workspace and the gateway.

8.6 A2A AGENT REGISTRY

Figure 8.6 shows the registry view. It lists the Agent Cards returned by the registry endpoint, with each agent's domain, supported tasks, skills and verification status, and the declared dependency edges.

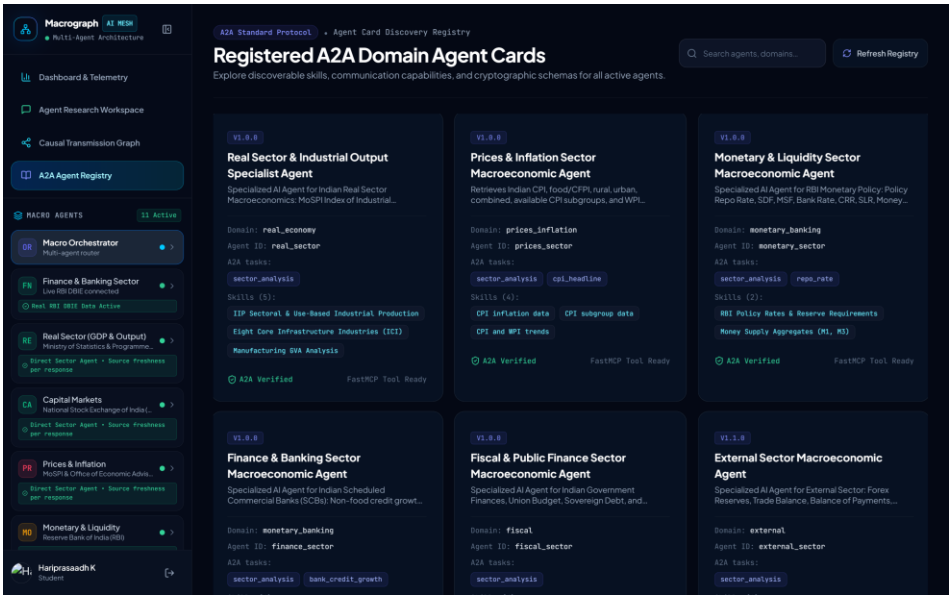


Figure 8.6: A2A agent registry view listing the Agent Cards of the sector agents.

8.7 CAUSAL LAB

The causal lab has five tabs, backed by the research endpoints described in Chapter 7. The path tab (Figure 8.7) queries the transmission path between any two indicators and lists the relations with their lag, sign, type and confidence.

The Shockwave tab animates how a shock spreads month by month through the graph, with adjustable magnitude and response shape (Figure 8.8). The Twin tab compares the baseline and shocked worlds and shows the attribution table. The Radar tab lists anomalies with their classification. The MPC tab runs the simulated committee and shows each seat's vote, the tally, the dissent and the rule baseline (Figure 8.9). Labels on every panel mark the data as simulated or cached.

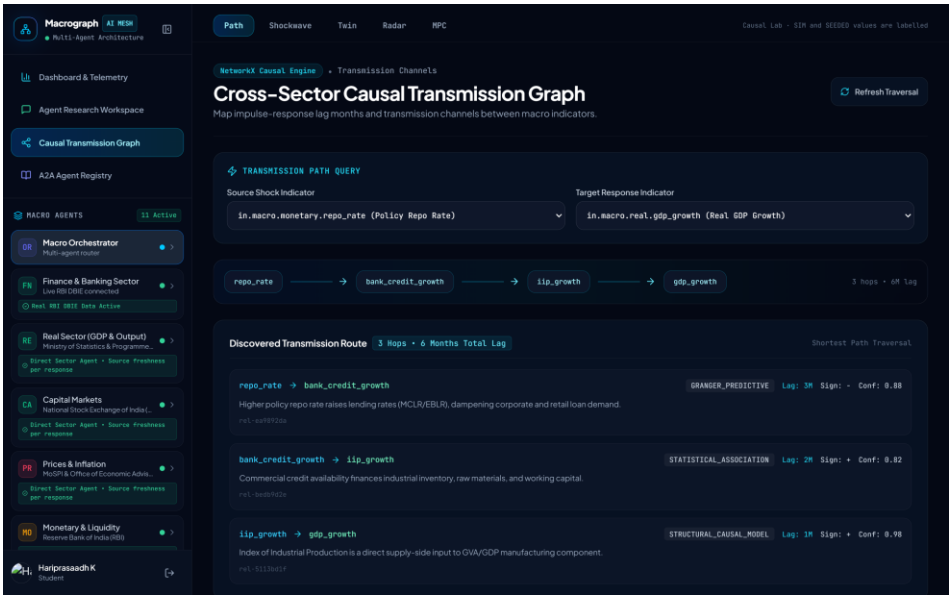


Figure 8.7: Causal lab, path tab: transmission path from the repo rate to GDP growth.

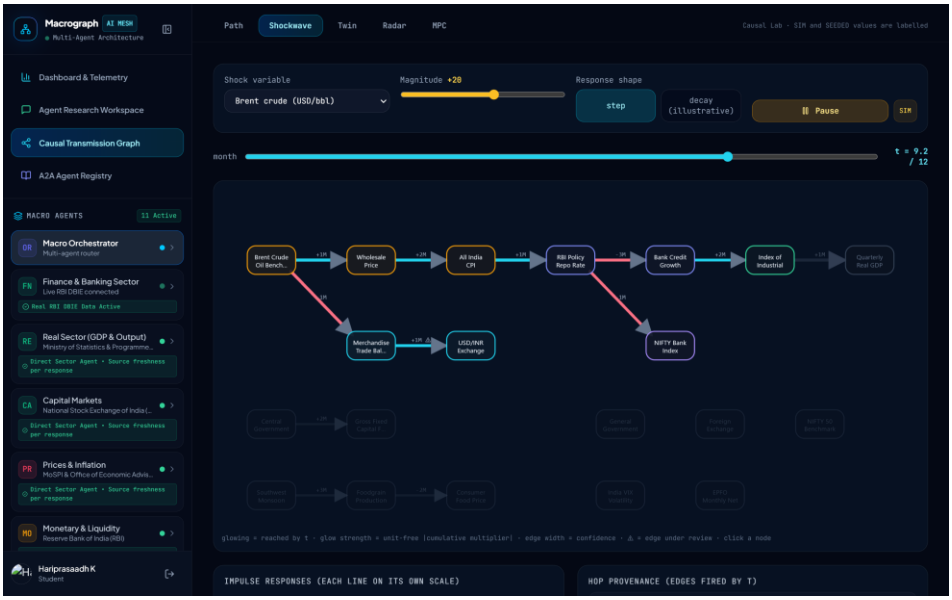


Figure 8.8: Causal lab, Shockwave tab: animated propagation of a Brent crude shock.

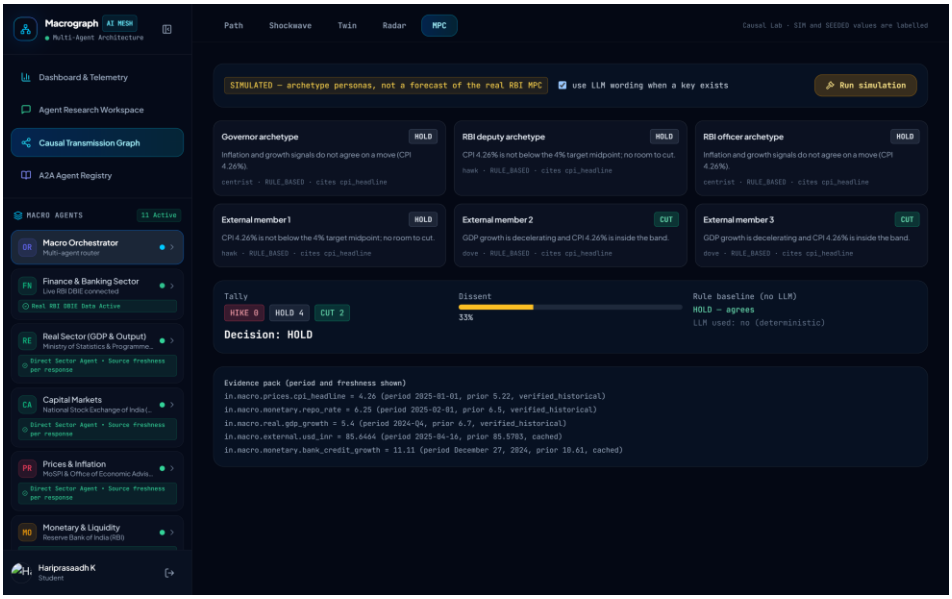


Figure 8.9: Causal lab, MPC tab: simulated six-seat committee labelled as simulated.

8.8 HTTP INTERFACE

Table 8.1 summarises the interface the gateway exposes. FastAPI generates interactive documentation. When keys are configured, the research and brief endpoints call paid language-model and search services, and the current version leaves them unauthenticated.

Table 8.1: Principal HTTP endpoints

Endpoint	Purpose
POST /api/v1/analyze	Full orchestrator run with optional scenario shock
POST /api/v1/chat/stream, POST /api/v1/chat	Streaming and synchronous chat with direct or orchestrated routing
POST /api/v1/simulate	Scenario shock through the knowledge graph
GET /api/v1/kg/path, GET /api/v1/kg/impacts	Transmission path and downstream impacts
GET /api/v1/dashboard/overview	Cross-sector telemetry with citations
GET /api/v1/research/kg/graph, /edges/audit	Graph for display and edge evidence audit
POST /api/v1/research/trajectory, /compare	Shockwave trajectories and twin comparison
GET /api/v1/research/anomalies	Anomaly radar, optional simulated injection
POST /api/v1/research/mpc/simulate	Simulated committee
GET /api/v1/brief/today, POST /brief/run, GET /brief/history	Daily early-warning brief
GET /a2a/registry, /a2a/v1/agents, /capabilities, /traces	Agent Cards, capabilities and conversation traces
GET /health	Status, number of agents, graph size, Neo4j flag

Chapter 9
Testing, Results and Discussion

9.1 INTRODUCTION

This chapter reports how the system was tested and what the measurements show. All figures were obtained on 12 and 13 October 2026 by running the repository. Where a result exposes a weakness of the system, the weakness is stated. Planned evaluation that has not yet been carried out is described separately in Section 9.7 and is not presented as a result.

9.2 TESTING STRATEGY

Testing follows four layers. Unit tests cover parsers, validators, the claim verification library and sector clients and databases. Component tests cover the A2A protocol and its hardening, the knowledge-graph schema and engine, a mocked Neo4j store and the causal engine. Integration tests cover A2A delegation and streaming, the orchestrator with the knowledge graph, and the dashboard and chat interfaces, using stub handlers and a mocked language model. A final layer is a manual system demonstration with *demo_platform.py* and a sector validation script. Changes pass the quality gates of four kinds: automated tests, a scan of the engineering rules, the frontend build and the demo.

9.3 TEST RESULTS

The repository contains 455 test functions, which execute as 551 test cases after parametrisation. Figure 9.1 and Table 9.1 show their distribution across areas. The core and cross-cutting suite is the largest, followed by agriculture, external, monetary, services and finance.

Table 9.1: Test functions by area (12 October 2026)

Area	Test functions
Core and cross-cutting (A2A, orchestrator, knowledge graph, causal engine, verification, dashboard)	198
Agriculture sector	48
External sector	45
Monetary sector	43
Services sector	33
Finance sector	32
Capital markets sector	27
Fiscal sector	15
Labour sector	7
Real sector	7
Total	455

The complete run of all sector test directories took 11 minutes 55 seconds and produced **550 passed and 1 failed**. The failure is the monetary-sector test that checks the fallback from the policy-rate fetcher to the stance fetcher: it expects a repo rate of 5.0 from its stub but the agent returned 5.25, which means that the agent now prefers another source over the stubbed fallback. It is a drift between the test and the code, not an observed failure of the running system, and it is recorded as an open item. The default *pytest* invocation collects 374 tests, because the other sector test directories are not listed in the project's test paths; running all directories is required to obtain the figures above.

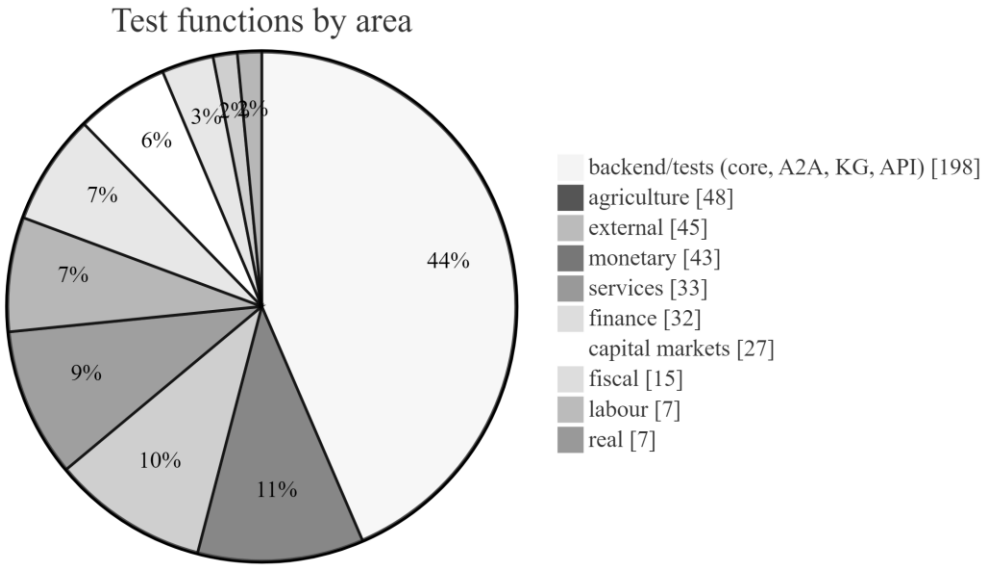


Figure 9.1: Number of test functions per test area (455 functions).

9.4 END-TO-END ORCHESTRATOR RUN

One cross-sector question was run through the full orchestrator with a crude oil scenario: “How does a rise in crude oil prices affect inflation and RBI repo rate policy?” with a shock of plus 20 US dollars per barrel on Brent crude. The run used live official sources and the deterministic offline mode (no language model key), and completed in about eleven seconds. The router selected the monetary, prices and agriculture agents. Table 9.2 lists the observations returned, and the report is reproduced in Appendix 3.

Table 9.2: Observations collected in the end-to-end run

Indicator	Value	Period	Source / status
CPI headline (YoY %)	5.54	September 2026	MoSPI, official
Repo rate (%)	5.25	February 2026	RBI DBIE
M3 growth (YoY %)	unavailable (None)	30 June 2026	RBI DBIE

The knowledge graph returned three transmission paths among the observed indicators: CPI to repo rate (1 hop, 1 month), Brent to CPI (2 hops, 3 months) and Brent to repo rate (3 hops, 4 months). The scenario used the live CPI and repo rate as baselines and the defaults for all other indicators. The report contained the indicator matrix with provenance hashes, a causal diagram, the path and scenario tables, a source provenance table and the statement that documentary evidence is unavailable. The run confidence score, which is the fraction of observations with usable status, was 1.0.

The run also exposed three defects. First, the M3 growth observation had no value, yet the indicator matrix printed it as “None%” instead of marking it unavailable. Second, the router included the agriculture agent for a question about crude oil, inflation and the repo rate, which shows that the keyword scoring is too permissive for that agent. Third, when no language model key is configured the prices agent logs a configuration error and continues with official observations; the behaviour is correct but the error log is misleading. These defects are recorded in the project's status document.

9.5 EDGE EVIDENCE AUDIT RESULT

The edge evidence audit was run on the series available in the running system. Table 9.3 and Figure 9.2 show the result: **none of the twelve edges could be tested**

empirically. Seven edges had fewer than ten overlapping months (between zero and two), and five had no series for one of their endpoints. All twelve edges therefore remain SEEDED, and the p-values stored in the ontology are priors that the platform cannot currently confirm or reject. This is the most important finding about the causal model: the graph is a documented set of assumptions, and the scenario outputs depend on them.

Table 9.3: Result of the edge evidence audit

Edge	Overlap (months)	Status
Brent crude to WPI	0	Untestable (too few months)
WPI to CPI headline	1	Untestable (too few months)
CPI headline to repo rate	1	Untestable (too few months)
Repo rate to bank credit growth	0	Untestable (too few months)
Bank credit growth to IIP growth	0	Untestable (too few months)
IIP growth to GDP growth	0	Untestable (no data)
Brent crude to trade balance	1	Untestable (too few months)
Trade balance to USD/INR	2	Untestable (too few months)
Repo rate to bank NIFTY	0	Untestable (no data)
Central capex to GFCF investment	0	Untestable (no data)
Monsoon departure to foodgrain production	0	Untestable (no data)
Foodgrain production to CPI food	0	Untestable (no data)

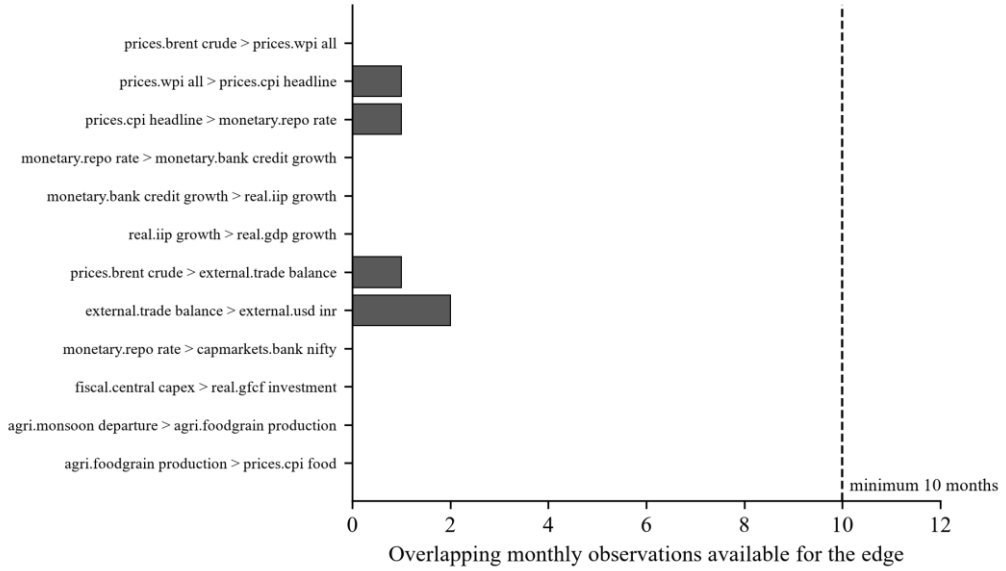


Figure 9.2: Overlapping monthly observations available for each causal edge (minimum required: 10).

9.6 ANOMALY RADAR AND SIMULATED COMMITTEE

The anomaly radar scanned 18 series. Ten of them were skipped because they had fewer than eight observations (between one and three points for GDP growth, IIP growth, CPI, WPI, Brent crude, the repo rate, debt-to-GDP, NIFTY 50, foodgrain production and EPFO additions). Of the eight series analysed, one exceeded the threshold: the merchandise trade balance for August 2024 with a z-score of −2.65. Its downstream neighbour, USD/INR, had a z-score of only 1.16, so the anomaly was classified as isolated and marked for source verification. The result shows that the radar works as designed but that the available history is too short for most indicators to be scanned.

The simulated committee (Table 9.4) voted four to two to hold, with two dovish external members voting to cut because GDP growth was decelerating while CPI was inside the band. The rule-only decision was the same (hold) and the dissent was 0.33. The evidence pack used the canonical store, whose latest CPI was 4.26 per cent for January 2025 and whose repo rate was 6.25 per cent for February 2025, while the end-to-end run above retrieved live values of 5.54 per cent and 5.25 per cent. The simulated meeting is therefore based on older data than the live agents, a weakness that must be removed before the simulator is evaluated against actual decisions.

Table 9.4: Votes of the simulated committee (rule-based mode)

Seat	Persona	Vote	Reason (rule)
Governor	Centrist	HOLD	Inflation and growth signals do not agree on a move (CPI 4.26%).
RBI deputy	Hawk	HOLD	CPI 4.26% is not below the 4% target midpoint; no room to cut.
RBI officer	Centrist	HOLD	Inflation and growth signals do not agree on a move (CPI 4.26%).
External member 1	Hawk	HOLD	CPI 4.26% is not below the 4% target midpoint; no room to cut.
External member 2	Dove	CUT	GDP growth is decelerating and CPI 4.26% is inside the band.
External member 3	Dove	CUT	GDP growth is decelerating and CPI 4.26% is inside the band.

9.7 PLANNED EVALUATION

The measurements above show that the system works, but they do not show that it is better than alternatives. The planned evaluation uses a question set of about 250 questions (point lookups, changes, cross-sector syntheses, causal and scenario

questions and unanswerable questions) whose gold answers are computed from frozen official data. Five systems are compared: a single language model, a model with web search, retrieval-augmented generation, the multi-agent system without verification and the multi-agent system with verification. The principal metrics are the unsupported numeric claim rate, citation validity, abstention correctness, coverage, latency and cost, reported as means with bootstrap intervals over repeated runs and at least two language model backends.

Ablations remove one mechanism at a time: the ownership constraint, claim verification, edge evidence grading, the knowledge graph, and the multi-agent decomposition. A second planned benchmark compares the simulated committee with actual decisions and vote splits using point-in-time data.

9.8 LIMITATIONS

The limitations of the current system are discussed here.

1. **Seeded edge statistics.** The edge confidences and p-values are curated priors; Section 9.5 shows that none can currently be tested. Scenario outputs depend on them.
2. **Heuristic and unit-blind scenario engine.** The constants are design parameters, deltas are in each indicator's own unit and the band is not a statistical interval. One edge sign is known to be doubtful.
3. **Short histories.** Many series have fewer than eight observations, which limits both the radar and the audit.
4. **Unprotected cost-bearing endpoints.** The research and brief endpoints have no authentication or rate limits.
5. **No formal evaluation yet.** The comparison with baselines is planned but not done.

9.9 DISCUSSION

The results support three conclusions. First, the engineering goals were met: ten sector agents, a typed protocol with guards and provenance, an orchestrator that degrades gracefully, and an interface that exposes sources and uncertainty. A single question produced a cited report in about eleven seconds with live data and no language model. Second, the platform is honest about what it does not know: the audit reports that no

edge can be tested, the radar reports which series are too short, and the simulated committee is labelled simulated. Third, the main scientific claims are not yet established. The causal graph rests on assumptions, the scenario engine is a heuristic, and the verification layer is not connected to the answers. Chapter 10 sets out the work needed to turn these into evaluated contributions.

Cha pter 10
Conclusion and Future Work

10.1 SUMMARY OF THE WORK

This project designed and implemented Macrograph AI, a multi-agent platform for Indian macroeconomic intelligence. The economy is decomposed into ten sector agents, each owning its indicators and obtaining them from official sources through its own MCP server. The agents cooperate through a native typed agent-to-agent layer that enforces declared dependencies, rejects loops and refuses successful responses without sources. A four-node LangGraph orchestrator routes questions over Agent Cards, collects observations, maps transmission paths in a causal knowledge graph of twenty indicators and twelve edges, runs an optional scenario, and produces a report in which every observation carries its period, status and a provenance hash. When data or a language model is unavailable the system degrades to cached values, structured unavailable statuses and deterministic summaries instead of invented figures. A web application provides a cited dashboard, streaming chat, an A2A registry and an interactive causal lab.

The project also produced research instruments: an edge evidence audit, an anomaly radar, trajectory and twin views, a simulated Monetary Policy Committee with a rule baseline, a daily early-warning brief and a claim verification library. The implementation comprises about 37,000 lines of Python and 11,300 lines of TypeScript and is covered by 455 test functions, of which the complete run passed 550 of 551 executed cases.

10.2 ACHIEVEMENT OF THE OBJECTIVES

Table 10.1 compares the delivered work with the objectives defined in Chapter 1.

Table 10.1: Status of the project objectives

Objective	Status	Evidence
O1 Sector-wise data access	Delivered	Ten MCP servers and local stores (Chapters 3 to 5)
O2 Coordinated multi-agent reasoning	Delivered	A2A layer with guards and trace; orchestrator; end-to-end run in Section 9.4
O3 Source-grounded answers	Delivered	Provenance enforced at protocol level; claim verification not yet in the answer path
O4 Explicit causal model	Delivered	Graph and scenario engine delivered; edge statistics are priors and the engine is heuristic
O5 Traceable provenance	Delivered	SourceProvenance, provenance hashes, conversation traces
O6 Research instruments	Delivered as prototypes	Audit, radar, trajectory, twin, MPC, brief; results in Sections 9.5 and 9.6
O7 Usable interface	Delivered	Dashboard, chat, causal lab and registry (Chapter 8)

10.3 CONTRIBUTIONS AND INSIGHTS

The engineering contribution is a complete, tested platform in which indicator ownership, protocol-level provenance and graceful degradation are built in rather than added afterwards. The most instructive results, however, are the honest findings. The edge audit shows that, with the data currently available, none of the twelve causal edges can be confirmed, which makes explicit that the knowledge graph is a set of documented assumptions. The worked example exposes a doubtful edge sign through an implausible exchange rate response. The end-to-end run exposes display and routing defects. Measuring and reporting these facts, instead of presenting the platform as finished, is consistent with the principle on which it was built: a statement without a source must not be presented as fact.

10.4 FUTURE WORK

The review of the implementation identified the following work, in order of priority. Table 10.2 gives an indicative schedule of about twelve weeks.

1. **Connect claim verification to answers.** Make the orchestrator's language model return structured claims, verify them against the A2A evidence, render only verified statements and report withheld claims, with causal claims hedged by the evidence class of their edge (Figure 10.1).

2. **Correct and estimate the causal model.** Correct the trade balance to USD/INR sign; replace typed confidences and p-values with estimates obtained from long monthly series using stationarity checks, Granger tests with false-discovery-rate control, local projections and stability tests; and report which seeded edges survive.
3. **Replace the scenario heuristic.** Estimate impulse responses with local projections or a Bayesian vector autoregression and evaluate them on episodes such as the 2022 oil price shock and the 2022-23 policy rate cycle, keeping the present engine as a baseline.
4. **Build the evaluation.** Implement the question set, baselines and ablations described in Chapter 9, and a point-in-time benchmark of the simulated committee against actual decisions.
5. **Add documentary evidence.** Index RBI minutes and the Economic Survey with passage-level attribution, or remove all retrieval language permanently.
6. **Harden the system.** Authentication and rate limits for cost-bearing endpoints, explicit start-up instead of import-time database initialisation, visible reporting of degraded components, persisted A2A traces, splitting of over-long modules, removal of *any* types and inline styles, and repair of the out-of-date test.
7. **Fix the observed defects.** Display unavailable values correctly, tighten routing for the agriculture agent and align the committee's evidence pack with the live data used by the agents.

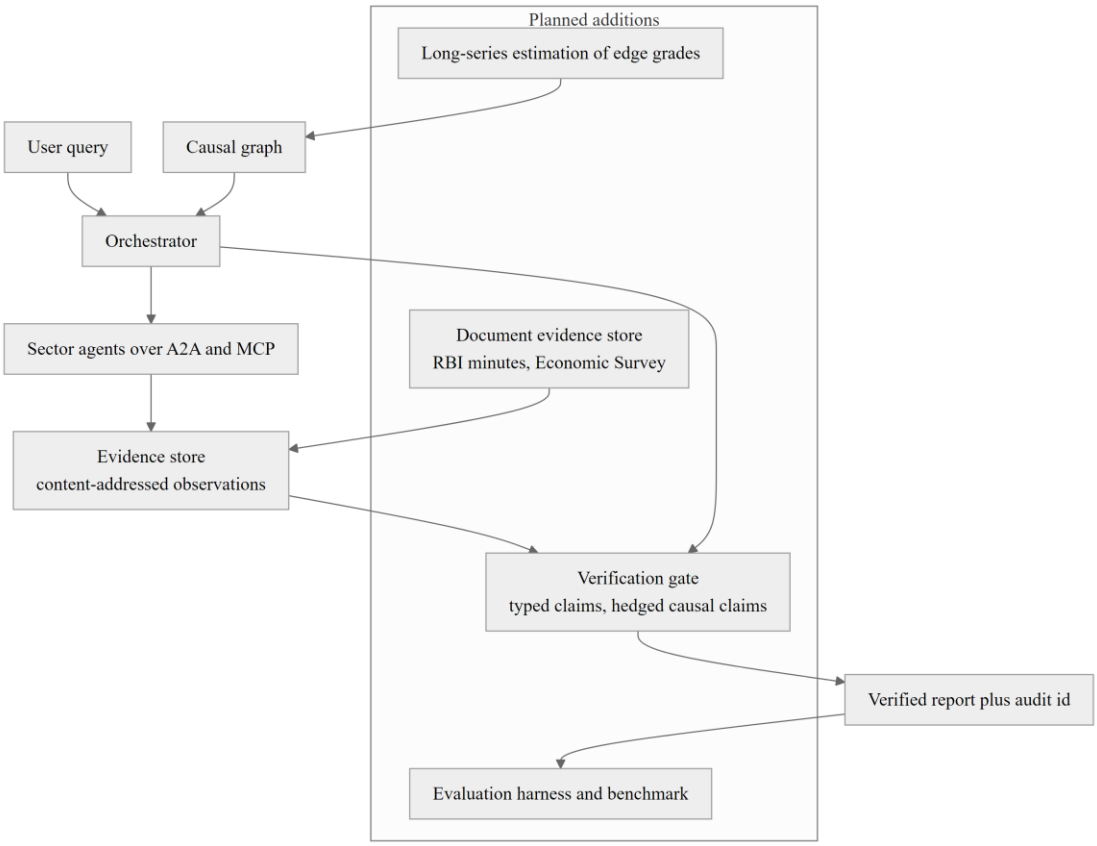


Figure 10.1: Target architecture after planned improvements.

10.5 CONCLUDING REMARKS

Macrograph AI shows that a multi-agent system for macroeconomic analysis can be built so that responsibility for each number lies with one agent, every number is accompanied by its source, and missing evidence is reported openly. It also shows, through its own audit, how much of the causal reasoning still depends on assumptions. The completed platform is a foundation on which the planned estimation, verification and evaluation can establish its analytical claims.

REFERENCES

(All references are in APA format)

[1]. Anthropic. (2024, November 25). *Introducing the Model Context Protocol*. https://www.anthropic.com/news/model-context-protocol

[2]. Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological), 57*(1), 289–300.

[3]. Dadopoulos, M., Ladas, A., Moschidis, S., & Negkakis, I. (2025). *Metadata-driven retrieval-augmented generation for financial question answering*. arXiv. https://arxiv.org/abs/2510.24402

[4]. Dickey, D. A., & Fuller, W. A. (1979). Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association, 74*(366), 427–431.

[5]. Google. (2025, April 9). *Announcing the Agent2Agent Protocol (A2A)*. Google Developers Blog. https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

[6]. Granger, C. W. J. (1969). Investigating causal relations by econometric models and cross-spectral methods. *Econometrica, 37*(3), 424–438.

[7]. Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference* (pp. 11–15).

[8]. Hao, Y., & Xie, D. (2025). *A multi-LLM-agent-based framework for economic and public policy analysis*. arXiv. https://arxiv.org/abs/2502.16879

[9]. Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., de Melo, G., Gutierrez, C., Kirrane, S., Gayo, J. E. L., Navigli, R., Neumaier, S., Ngomo, A.-C. N., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., & Zimmermann, A. (2021). Knowledge graphs. *ACM Computing Surveys, 54*(4), Article 71. https://doi.org/10.1145/3447772

[10]. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys, 55*(12), Article 248. https://doi.org/10.1145/3571730

[11]. Jordà, Ò. (2005). Estimation and inference of impulse responses by local projections. *American Economic Review, 95*(1), 161–182.

[12]. Kato, M. (2026). *AI economist agent: An agentic framework for model-grounded economic analysis with RAG, knowledge graphs, and large language models*. arXiv. https://arxiv.org/abs/2606.20041

[13]. LangChain. (n.d.). *LangGraph documentation*. Retrieved October 12, 2026, from https://langchain-ai.github.io/langgraph/

[14]. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474.

[15]. Lin, J., Sun, L., & Yan, Y. (2026). *Simulating macroeconomic expectations in survey experiments with LLM-based economic agents*. arXiv. https://arxiv.org/abs/2505.17648

[16]. Lowin, J. (n.d.). *FastMCP* [Computer software]. GitHub. https://github.com/jlowin/fastmcp

[17]. Ministry of Statistics and Programme Implementation. (n.d.). *e-Sankhyiki: MoSPI data and statistics services*. Government of India. https://mcp.mospi.gov.in/

[18]. Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (Article 2). ACM. https://doi.org/10.1145/3586183.3606763

[19]. Pearl, J. (2009). *Causality: Models, reasoning, and inference* (2nd ed.). Cambridge University Press.

[20]. Pydantic Services Inc. (n.d.). *Pydantic documentation*. Retrieved October 12, 2026, from https://docs.pydantic.dev/

[21]. Raasveldt, M., & Mühleisen, H. (2019). DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data* (pp. 1981–1984). ACM. https://doi.org/10.1145/3299869.3320212

[22]. Ramírez, S. (n.d.). *FastAPI* [Computer software]. https://fastapi.tiangolo.com/

[23]. Reserve Bank of India. (n.d.). *Database on Indian Economy (DBIE)*. https://data.rbi.org.in/

[24]. Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 92–96).

[25]. Sims, C. A. (1980). Macroeconomics and reality. *Econometrica, 48*(1), 1–48.

[26]. Wooldridge, M., & Jennings, N. R. (1995). Intelligent agents: Theory and practice. *The Knowledge Engineering Review, 10*(2), 115–152.

[27]. Xiao, Y., Sun, E., Luo, D., & Wang, W. (2025). *TradingAgents: Multi-agents LLM financial trading framework*. arXiv. https://arxiv.org/abs/2412.20138

[28]. Yang, Y., Pang, Y., Huang, G., & E, W. (2020). *The knowledge graph for macroeconomic analysis with alternative big data* (SSRN Scholarly Paper No. 3707964). Social Science Research Network. https://ssrn.com/abstract=3707964

[29]. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629

[30]. Zheng, S., Trott, A., Srinivasa, S., Parkes, D. C., & Socher, R. (2022). The AI Economist: Taxation policy design via two-level deep multiagent reinforcement learning. *Science Advances, 8*(18), Article eabk2607. https://doi.org/10.1126/sciadv.abk2607

[31]. Zhu, X., Xie, Y., Liu, Y., Li, Y., & Hu, W. (2025). *Knowledge graph-guided retrieval augmented generation*. arXiv. https://arxiv.org/abs/2502.06864

APPENDICES

Appendices are provided to give supplementary information that is not included in the main text. They are numbered with Arabic numerals (Appendix 1, Appendix 2, and so on); tables, figures and listings in them carry the appendix number and are referred to from the main text.

APPENDIX 1: REPOSITORY STRUCTURE AND CONFIGURATION

Figure A1.1 shows the layout of the repository, and Table A1.1 lists the configuration variables read from *backend/.env*.

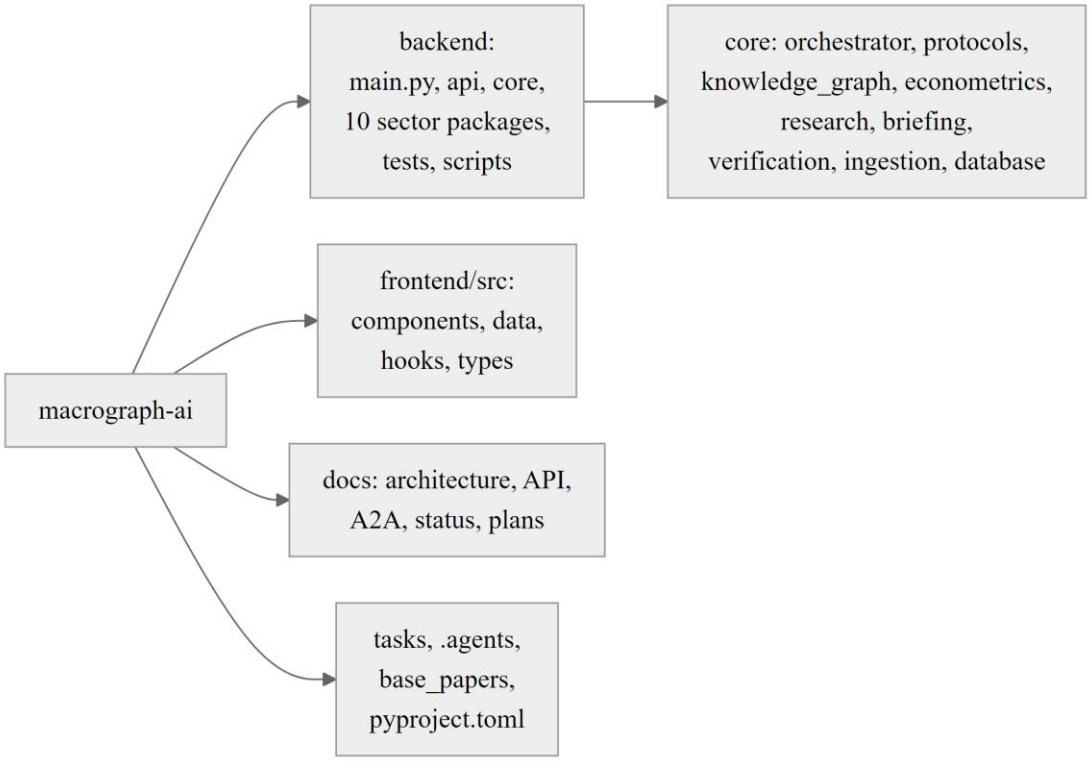


Figure A1.1: Repository layout of Macrograph AI.

Table A1.1: Principal configuration variables

Variable	Purpose
AGR_REAL_KEY, FIN_FIS_KEY, CAP_MON_KEY, PRIC_LAB_KEY, SERV_EXT_KEY	Language model keys, each shared by a pair of sector agents
ORCH_KEY	Language model key of the orchestrator
FAST_MODEL, REASONING_MODEL	Model identifiers for fast and reasoning calls (defaults openai/gpt-oss-20b and openai/gpt-oss-120b)
TVLY_KEY_1 to TVLY_KEY_3	Search keys for news retrieval
A2A_MAX_DEPTH, A2A_MAX_HOPS	Loop protection limits (defaults 5 and 25)
A2A_REQUEST_TIMEOUT, A2A_PEER_TIMEOUT	Timeouts in seconds (defaults 90 and 25)
A2A_API_KEY, A2A_ALLOW_ANONYMOUS_REQUESTS	Access control of the A2A HTTP interface
NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD	Optional graph mirror (disabled when the URI is empty)
BRIEF_CRON_ENABLED	Enables the 08:15 daily brief scheduler
CORS_ORIGINS	Allowed browser origins (default local development origins)

APPENDIX 2: CANONICAL INDICATORS OF THE KNOWLEDGE GRAPH

Table A2.1 lists the twenty canonical indicators registered in the ontology on 12 October 2026.

Table A2.1: Canonical indicators of the ontology

Identifier	Name	Domain	Unit	Frequency
in.macro.real.gdp_growth	Quarterly real GDP growth rate	Real economy	% YoY	Quarterly
in.macro.real.iip_growth	Index of Industrial Production growth	Real economy	% YoY	Monthly
in.macro.real.gfcf_investment	Gross fixed capital formation	Real economy	% of GDP	Quarterly
in.macro.prices.cpi_headline	All India CPI combined headline inflation	Prices	% YoY	Monthly
in.macro.prices.cpi_food	Consumer food price index inflation	Prices	% YoY	Monthly
in.macro.prices.wpi_all	Wholesale price index, all commodities	Prices	% YoY	Monthly
in.macro.prices.brent_crude	Brent crude oil benchmark price	Prices	USD/barrel	Daily
in.macro.monetary.repo_rate	RBI policy repo rate	Monetary	% p.a.	Policy cycle
in.macro.monetary.bank_credit_growth	Bank credit growth	Monetary	% YoY	Fortnightly
in.macro.fiscal.debt_to_gdp	General government debt to GDP ratio	Fiscal	% of GDP	Annual
in.macro.fiscal.central_capex	Central government capital expenditure	Fiscal	INR crore	Monthly
in.macro.external.forex_reserves	Foreign exchange reserves of India	External	Billion USD	Weekly
in.macro.external.usd_inr	USD/INR exchange rate	External	INR/USD	Daily
in.macro.external.trade_balance	Merchandise trade balance	External	Billion USD	Monthly
in.macro.capmarkets.nifty_50	NIFTY 50 benchmark index	Capital markets	Points	Daily
in.macro.capmarkets.bank_nifty	NIFTY Bank index	Capital markets	Points	Daily
in.macro.capmarkets.india_vix	India VIX volatility index	Capital markets	Points	Daily
in.macro.agri.monsoon_departure	Southwest monsoon rainfall departure from LPA	Agriculture	% departure	Seasonal
in.macro.agri.foodgrain_production	Foodgrain production volume	Agriculture	Million tonnes	Annual
in.macro.labour.epfo_additions	EPFO monthly net payroll additions	Labour	Thousand	Monthly

APPENDIX 3: SAMPLE REPORT GENERATED BY THE ORCHESTRATOR

The following tables reproduce the structured sections of the report produced by the end-to-end run of Section 9.4 (query: how does a rise in crude oil prices affect inflation and RBI repo rate policy, with a Brent crude shock of plus 20 USD per barrel; deterministic mode, live official sources, retrieved on 12 October 2026). The executive synthesis of this run read: "LLM narrative unavailable; deterministic summary of retrieved evidence follows. Observations retrieved from sector agents: 3. Causal transmission paths mapped in the knowledge graph: 3. Scenario simulated: Crude +20 on in.macro.prices.brent_crude. Documentary evidence: unavailable (no document store is configured); no quotations are provided."

Table A3.1, Table A3.2 and Table A3.3 reproduce the structured tables of the report.

Table A3.1: Indicator matrix of the generated report

Indicator	Value	Unit	Period	Data Status	Provenance Hash
in.macro.monetary.repo_rate	5.25	%	Feb 2026	RBI DBIE	37e0e36c25c48aea
in.macro.monetary.m3_growth	None%	% YoY	2026-06-30	RBI DBIE	37e0e36c25c48aea
in.macro.prices.cpi_headline	5.54	% YoY	2026-09	official	fed3aaaa910e5b26

Table A3.2: Scenario table of the generated report (live baselines for CPI and repo rate)

Target Indicator	Baseline	Shocked Peak	Impact (Delta)	Transmission Lag	Confidence Band
Wholesale Price Index All Commodities	2.45	4.02	+1.57	1M	[3.63, 4.41]
Merchandise Trade Balance	-22.5	-24.02	-1.52	1M	[-24.4, -23.64]
All India CPI Combined Headline Inflation	5.54	6.32	+0.78	3M	[6.12, 6.52]
USD/INR Exchange Rate	86.4	85.68	-0.72	2M	[85.5, 85.86]
RBI Policy Repo Rate	5.25	5.66	+0.41	4M	[5.56, 5.76]
Bank Credit Growth	13.8	13.6	-0.20	7M	[13.55, 13.65]
NIFTY Bank Index	51500.0	51499.81	-0.19	5M	[51499.76, 51499.86]
Index of Industrial Production Growth	4.2	4.11	-0.09	9M	[4.09, 4.13]
Quarterly Real GDP Growth Rate	5.4	5.35	-0.05	10M	[5.34, 5.36]

Table A3.3: Source provenance table of the generated report

Agent	Source	Dataset / Table	Period	Retrieved
monetary_sector	Monetary & Liquidity Sector Macroeconomic A...	Monetary Sector Analysis	n/a	2026-10-12 18:48:45
prices_sector	National Statistical Office (NSO), MoSPI	CPI / CPI:2024:division=0:state=1:sector=3:...	2026-09	2026-10-12 18:48:49
agriculture_sector	Agriculture Agent	Agriculture findings	n/a	2026-10-12 18:48:42

APPENDIX 4: SELECTED SOURCE CODE LISTINGS

The listings below are extracted from the repository without modification (long lines may wrap): Listing A4.1, Listing A4.2, Listing A4.3 and Listing A4.4.

Listing A4.1: A2A request and response models (core/protocols/a2a/messages.py)

```
class A2ARequest(BaseModel):
    """One agent-to-agent task request."""
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True,
frozen=True)

    request_id: str = Field(default_factory=lambda: new_id("req"),
min_length=1)
    sender_agent: str = Field(..., min_length=1)
    receiver_agent: str = Field(..., min_length=1)
    task: str = Field(..., min_length=1)
    parameters: dict[str, Any] = Field(default_factory=dict)
    context: Optional[dict[str, Any]] = None
    conversation_id: str = Field(..., min_length=1)
    parent_request_id: Optional[str] = None
    depth: int = Field(default=0, ge=0)
    call_chain: tuple[str, ...] = Field(default_factory=tuple)
    timeout_seconds: Optional[float] = Field(default=None, gt=0)
    timestamp: datetime = Field(default_factory=_now)

    @model_validator(mode="after")
    def _no_self_parent(self) -> "A2ARequest":
        if self.parent_request_id == self.request_id:
            raise ValueError("parent_request_id must differ from
request_id")
        return self


class A2AResponse(BaseModel):
    """Result of an A2ARequest; failures are data, never raised across
agents."""
    model_config = _STRICT

    request_id: str
    sender_agent: str
    receiver_agent: str
    status: Literal["success", "partial", "failed"]
    result: Optional[dict[str, Any] | list[Any] | str] = None
    sources: list[SourceProvenance] = Field(default_factory=list)
    errors: list[A2AErrorInfo] = Field(default_factory=list)
    follow_ups: list[FollowUp] = Field(default_factory=list)
    metadata: dict[str, Any] = Field(default_factory=dict)
    timestamp: datetime = Field(default_factory=_now)

    @property
    def ok(self) -> bool:
        return self.status in ("success", "partial")

    @property
    def error_codes(self) -> list[ErrorCode]:
        return [e.code for e in self.errors]
```

Listing A4.2: Shortest transmission path query (core/knowledge_graph/networkx_engine.py)

```
    def get_shortest_transmission_path(self, source_id: str, target_id:
str) -> List[CausalRelationship]:
        """Finds the most direct transmission chain between two
macroeconomic indicators."""
```

```
        if not self.graph.has_node(source_id) or not
self.graph.has_node(target_id):
            return []

        try:
            path_nodes = nx.shortest_path(self.graph, source=source_id,
target=target_id)
        except (nx.NetworkXNoPath, nx.NodeNotFound):
            return []

        relationships: List[CausalRelationship] = []
        for u, v in zip(path_nodes[:-1], path_nodes[1:]):
            edge_data = self.graph.get_edge_data(u, v)
            if edge_data:
                # Take the highest confidence edge if multiple exist
                best_rel = max(edge_data.values(), key=lambda d:
d.get("confidence", 0.0))["relation"]
                relationships.append(best_rel)

        return relationships
```

Listing A4.3: Scenario propagation loop (core/econometrics/causal_engine.py)

```
        for impact in downstream_impacts:
            target_id = impact["target_indicator_id"]
            path =
self.graph.get_shortest_transmission_path(shock_variable, target_id)
            if not path:
                continue

            target_base = baselines.get(target_id, 5.0)
            # Calculate cumulative elasticity and attenuation
            cum_multiplier = 1.0
            for r in path:
                traversed_relations.append(r.relation_id)
                sign = 1.0 if r.elasticity_sign == "+" else -1.0
                # Attenuation coefficient per hop based on confidence
                decay = 0.55 * r.confidence_score
                cum_multiplier *= (sign * decay)

            delta = round(shock_magnitude * cum_multiplier * 0.15, 2)
            shocked_target = round(target_base + delta, 2)
            ci_low = round(shocked_target - abs(delta * 0.25), 2)
            ci_high = round(shocked_target + abs(delta * 0.25), 2)

            forecasts[target_id] = {
                "indicator_name": impact["target_name"],
                "sector": impact["sector"],
                "baseline": target_base,
                "shocked_peak": shocked_target,
                "delta": delta,
                "confidence_band": [ci_low, ci_high],
                "transmission_lag_months": impact["total_lag_months"],
                "confidence_score": impact["cumulative_confidence"],
                "mechanism_summary": " ->
".join(impact["transmission_chain"])
            }
```

Listing A4.4: Rolling z-score of the anomaly radar (core/research/anomaly.py)

```
def zscore(values: List[float]) -> Optional[float]:
    """z of the latest point against the trailing window, latest excluded
from its own baseline."""
    n = len(values)
    if n < MIN_N:
        return None
```

```
    window = min(WINDOW, n - 1)
    base = values[-window - 1:-1]
    mean = sum(base) / len(base)
    sd = (sum((x - mean) ** 2 for x in base) / len(base)) ** 0.5
    if sd == 0:
        return None
    return (values[-1] - mean) / sd
```

APPENDIX 5: ERROR TAXONOMY AND OPEN ISSUES

Figure A5.1 groups the A2A error codes. Table A5.1 lists the principal open issues identified in the review of 12 October 2026 and referred to in Chapters 9 and 10.

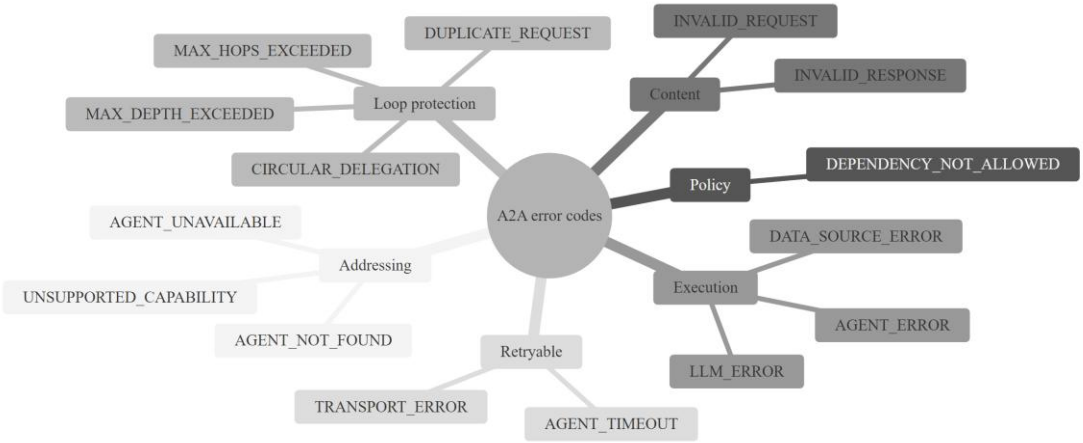


Figure A5.1: Taxonomy of A2A error codes.

Table A5.1: Principal open issues from the code review

Issue	Severity	Planned action
Claim verification library is not called by any answer path	High	Integrate into synthesis
Edge p-values and confidences are curated priors	High	Estimate from long series; audit
Scenario engine is heuristic, unit-blind, with fixed default baselines	High	Local projections or BVAR
Trade balance to USD/INR edge has a doubtful sign	High	Correct sign and test
Optional sector mounts and some start-up steps fail silently	High	Log and surface degradation
Edge audit returns an empty result on any exception	Medium	Surface the error
Constants duplicated in engine and research module	Medium	Single shared definition
Private language model client members used by callers	Medium	Public availability property
Granger audit lacks stationarity checks and multiple-testing control	Medium	Differencing, BH correction
Research and brief endpoints unauthenticated	Medium	Authentication and rate limits
Sector test directories missing from default test paths	Medium	Add to configuration; fix drifted test
Nine non-test modules exceed 400 lines; frontend has any types and inline styles	Low	Refactor